応用情報技術者試験　テクノロジ系　#5

ソフトウェア

用語集まとめ ― 過去問33回分（平成21～令和7年度）を読み込んだ参考書版



点線の下線の用語・青い過去問番号・各ページ下の「詳しく:」を押すと、参照元のサイト（e-Words／応用情報技術者試験.com）で詳しい解説が開く

全体像 シラバスの5つの小分類を、5章に分けて読む

1 OSの種類と構成
基本ソフトウェア・カーネル・動作モード・ジョブ管理・スプーリング・起動・アクセス制御

2 タスク管理
3つの状態・プリエンプション・スケジューリング・多重度・排他制御・デッドロック

3 記憶管理
区画方式・断片化・仮想記憶・ページフォールト・ページ置換え（FIFO・LRU）

4 ミドルウェアとファイルシステム
TPモニター・API・ライブラリ・シェル・パス・リンク・バックアップ

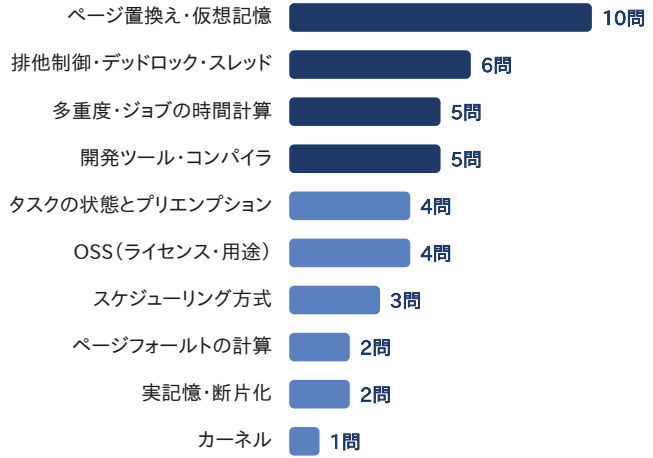
5 開発ツールとOSS
コンパイラの処理・リンカとローダ・デバッグの道具・Git・ライセンス・用途別のOSS

午前での出方	
問題数	毎回 **問16〜問20 あたりで4問前後**（R7春は問14〜17）。10回で42問
同じ問題の再出題	5組。プリエンプションの遷移先・多重度の計算・デッドロックの防止・スラッシング・Git。I2Cのセマフォは選択肢を1つ変えて再出題
問題の型	**時間の線や表を書いて数える計算が約3分の1**（ターンアラウンド・多重度・ページ置換え）。残りは用語の入れ替えを見抜く問題

この資料の読み方：1枚に1つのメッセージ。計算の問題は、図と同じ線や表を自分で書けば解けるようにしてある。

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム ／ 応用情報.com 用語集 5-2 ミドルウェア

4分の1は仮想記憶。ページの出し入れを表に書ければ取れる

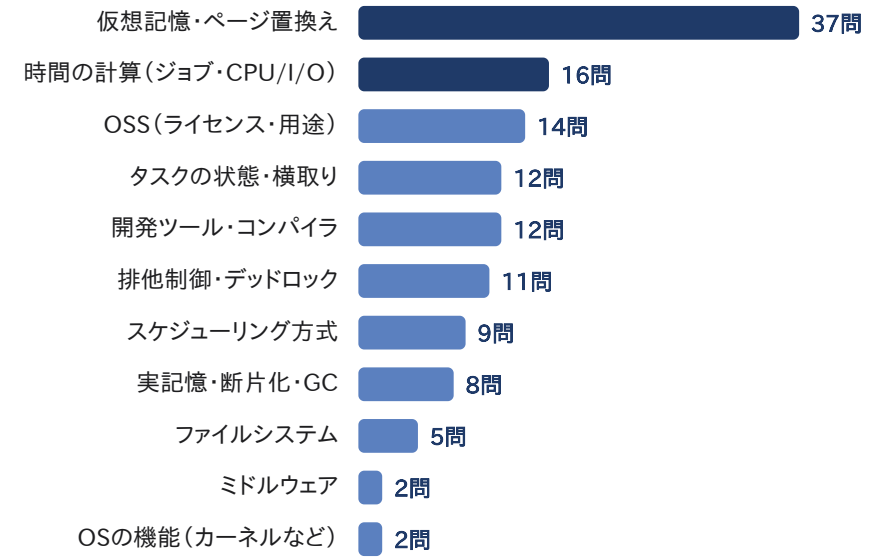


令和3年度春期〜令和7年度秋期の42問を、論点ごとに数えた（同じ問題も1問ずつ数えた）

論点	押さえること
ページ置換え	FIFO＝読み込んだ時刻、LRU＝最後に使った時刻、LFU＝回数。**見る列が違う**
仮想記憶の事象	ページフォールト＝ページイン ≧ ページアウト。急に遅くなるのはスラッシング
排他制御	共有資源はセマフォ。デッドロックは**資源を取る順番をそろえて**防ぐ
時間の計算	横軸に時間を取った線（ガントチャート）を書き、終了時刻－到着時刻
状態遷移	横取りされたら**実行可能状態**。待ち状態ではない
開発ツール・OSS	道具の名前と役目の組合せ。GPL→LGPL→MPL→BSDの順に義務が軽い

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム ／ 応用情報.com 用語集 5-5 オープンソースソフトウェア

17年・128問で見ても、仮想記憶と「時間の計算」で4割を超える



平成21年度春期～令和7年度秋期の33回、この分野の128問を論点ごとに数えた（同じ問題も1問ずつ）

何度も出た問題（平成21～令和7年度）

論点	出た回数
スラッシングの名前を答える	**6回**（H23特・H24春・H27春・H29秋・R3秋・R6秋）
デッドロックの起きる組合せ（表）	**4回**（H23秋・H26秋・R2秋・R6春）
空きが最大の媒体にファイル領域を割当て	**4回**（H22春・H24秋・H27秋・R1秋）
ラウンドロビン方式の説明	**4回**（H25秋・H26春・H27春・H29秋）
プリエンプションの契機・遷移先	**5回**（H27秋・H29秋・H30秋・R3春・R7春）
ページインだけの処理の割合	**3回**（H28春・H30秋・R5秋）

読み方：平成の問題は、数字や選択肢の順を変えて令和でも出ている。この資料の例題は、平成の過去問も使って**同じ型を2回以上**解けるようにした。

出典：IPA 応用情報技術者試験 午前 過去問題（平成21年度春期～令和7年度秋期）をClaudeが分類して数えた

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

第1章

OSの種類と構成

ソフトウェアの体系 ／ カーネル ／ CPUの動作モード ／ ジョブ管理 ／ スプーリング ／ 起動 ／ 利用者とアクセス制御



詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

OSはハードウェアとアプリの間で、資源を配る係。中核がカーネル

応用ソフトウェア
販売管理・表計算・Webブラウザ

ミドルウェア
DBMS・TPモニター・Webサーバ・運用管理ツール（第4章）

OS（基本ソフトウェア）
制御プログラム（中心がカーネル）＋言語プロセッサ＋サービスプログラム

ハードウェア
CPU・主記憶・ディスク・入出力装置

> OSとミドルウェアをまとめて**システムソフトウェア**と呼ぶ。サービスプログラム（整列・併合・アーカイバなど）は**ユーティリティ**とも呼ぶ。

OSの管理機能	受け持つこと
ジョブ管理	利用者の仕事（ジョブ）を受け付け、順番を決めて実行させる
タスク管理	実行の単位（タスク）にCPUを割り当てる（第2章）
記憶管理	主記憶を割り当て、仮想記憶を管理する（第3章）
データ管理	ディスク上のデータをファイルとして読み書きさせる
入出力管理	装置の割当てとスプーリング・バッファリング
運用管理・障害管理	起動と終了・利用者の管理・監視・障害の記録と回復

> OSの種類：**リアルタイムOS**は締切りまでに応答することを最重視。**組込みOS**は機器に入る小さなOS。**セキュアOS**はrootの権限も細かく制限する。

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム ／ 応用情報.com 用語集 5-2 ミドルウェア

⏱️ リアルタイムOSは「速さ」より「締切りを守ること」。だから作りが単純

観点	汎用OS（PC・サーバ）	リアルタイムOS（組込み）
目標	全体の**処理量**（スループット）と使いやすさ	決められた時間内に**必ず応答**する
スケジューリング	時分割（ラウンドロビン系）で公平に	**優先度順のプリエンプティブ**が基本
タスクの作り方	実行中に次々と生成・消滅	起動時に**静的に生成**しておくことが多い
仮想記憶	使う（MMUで保護も）	使わないことが多い（ページフォールトで時間が読めなくなる）
規模	大きい。ファイル管理・GUIも持つ	小さい。必要な機能だけを組み込む
例	Windows・macOS・Linux	μITRON（TOPPERS）・FreeRTOS・VxWorks

過去問 H21秋 問18「制御系の組込みシステムで使うリアルタイムOSの特徴」の正解は「**タスクの生成は主に静的に行う**」。

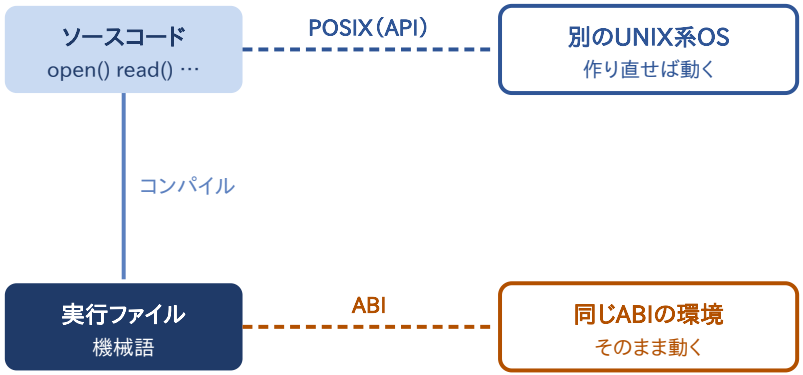
誤りの選択肢	どこが違うか
MMUで仮想記憶制御を行わなくてはならない	必須ではない。時間が読めなくなるので使わない方が多い
ファイルマネージャとメモリ保護は必須	機器に要るものだけ入れる。必須ではない
ラウンドロビンで全体のスループットを上げる	それは汎用OSの考え方。RTOSは優先度で締切りを守る

日本の家電・車載機器では TRON の流れをくむ **μITRON** 仕様のOSが広く使われてきた。TOPPERS はその仕様に沿った OSS の実装。

出典：IPA シラバス Ver.7.2 中分類5 1.（1）／IPA 平成21年度秋期 午前 問18／TOPPERSプロジェクト

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム ／ 応用情報.com 用語集 5-5 オープンソースソフトウェア

🤝 互換性は2段階。POSIXはソースの約束、ABIは機械語の約束



ソース互換＝作り直しが要る／バイナリ互換＝作り直し不要

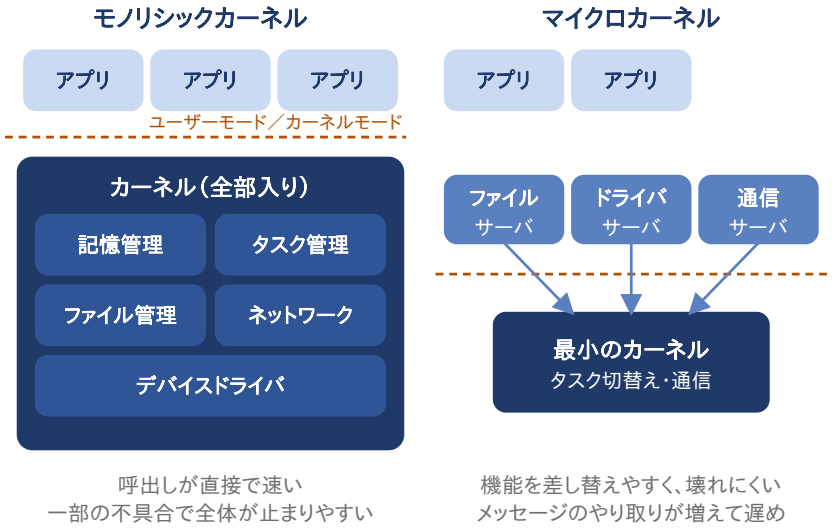
用語	意味
POSIX	UNIX系OSのAPI（関数・コマンド・シェル）の標準。IEEEと The Open Group が定める。従えばソースのまま他のOSへ移しやすい
ABI	機械語の段階の約束（呼出し規約・データの並べ方・システムコール番号）。同じなら**再コンパイルなし**で動く
OSE	Open Systems Environment。標準に従い、異なるメーカーの機器・ソフトを組み合わせられる環境
The Open Group	POSIX を含む UNIX の標準（Single UNIX Specification）を管理する団体

> **例：**Linux 用の C プログラムは、POSIX の範囲で書けば FreeBSD でもコンパイルし直して動く。だが Linux の実行ファイルをそのまま FreeBSD で動かすには、Linux の ABI をまねる仕組みが要る。

出典：IPA シラバス Ver.7.2 中分類5 1.（1）（13）・5.（2）の用語例／The Open Group「POSIX.1-2024（IEEE Std 1003.1）」

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム ／ 応用情報.com 用語集 5-5 オープンソースソフトウェア

カーネルに何を入れるかで2通り。シェルもGUIもカーネルの外

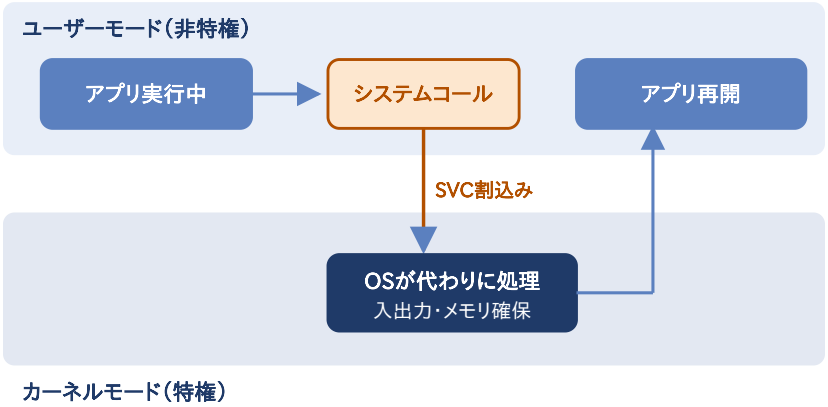


ソフトウェア	カーネルの中か外か
プロセス管理・メモリ管理	**中**。アプリが動くための基本機能
シェル（bashなど）	外。コマンドを解釈してカーネルを呼ぶ
デスクトップ環境（GUI）	外。画面の操作を受け持つ別のソフト
ブラウザ・ワープロ	外。応用ソフトウェア

R5秋 問19「Linuxカーネルの説明」：正解は「プロセス管理やメモリ管理などの基本機能を提供する」。シェルやGUIが**組み込まれている**とする選択肢は誤り。Linuxはモノリシックカーネル。

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム ／ 応用情報.com 用語集 5-5 オープンソースソフトウェア

🔑 アプリは特権命令を使えない。OSに頼むとカーネルモードに切り替わる



モード	動くもの	使える命令
カーネルモード（スーパーバイザー・特権）	OSのカーネル	全部。入出力・割込みの設定などの**特権命令**も
ユーザーモード（非特権）	アプリケーション	特権命令以外。主記憶も自分の範囲だけ

> アプリが自分でカーネルモードに上がることはできない。ユーザーモードで特権命令を実行すると**例外（プログラム割込み）**になり、実行されない。

> 目的は**保護**。アプリの誤りや悪意が、OSや他のプログラムを壊さないようにする。割込みの種類（SVC・入出力・タイマなど）は #3 にまとめた。

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

🔑 ファイルを1つ読むだけでも、アプリは何度もシステムコールでOSに頼む

Linux で「cat memo.txt」を実行したときの流れ（概略）

1. **openat**：「memo.txt を読みたい」→ OSが権限を調べ、**ファイル記述子**（番号）を返す
2. **read**：記述子の番号で中身を読む。OSがディスク（またはキャッシュ）からアプリの領域へ写す
3. **write**：読んだ内容を、標準出力（記述子1番）へ書く
4. **close**：記述子を返す。最後に **exit_group** で終了を伝える

> アプリは装置を直接さわれない（ユーザーモード）。すべて**システムコール**（SVC割込み）でカーネルモードの OS に代わりにやってもらう。

システムコールの例	頼む内容
fork・execve	プロセスを複製する・別のプログラムに入れ替える
openat・read・write	ファイルや装置を開く・読む・書く
mmap・brk	メモリ（仮想アドレス空間）を割り当てる
socket・connect	ネットワークの通信口を作る・つなぐ
kill	他のプロセスにシグナルを送る

数の目安 Linux の man ページ syscalls(2) には**数百**のシステムコールが並ぶ。アプリは普通、C ライブラリの関数（fopen など）経由で呼ぶので、自分で番号を書くことはない。

出典：Linux man-pages「syscalls(2)」／IPA シラバス Ver.7.2 中分類5 1.（2）（4）の用語例（SVC割込み・カーネルモード）

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

🔔 割込みが来たら、今の状態を保存→原因別の処理→ディスパッチの順に進む

割込み発生 CPUが命令の区切りで気づく ▶ **状態の保存** PC・レジスタ・PSW ▶ **割込み処理** 原因ごとのルーチン ▶ **ディスパッチ** 次に動くタスクへ

種類	原因	例
内部割込み（実行中の命令が原因）	SVC（スーパーバイザーコール）	システムコールでOSに頼む
	プログラム割込み	0での割り算・桁あふれ・特権命令の違反・ページフォールト
外部割込み（命令の外が原因）	入出力割込み	入出力が終わった（入出力終了割込み）
	タイマー割込み	タイムクウォンタムを使い切った
	機械チェック割込み	電源の異常・ハードウェアの故障

考え方 タスクの状態遷移は、ほとんどが割込みをきっかけに起きる。
- 入出力を頼む（SVC）→ 実行状態から**待ち状態**
- 入出力が終わる（入出力割込み）→ 待ちから**実行可能状態**
- 時間切れ（タイマー割込み）→ 実行から**実行可能状態**

ページフォールトは**内部**割込み（プログラム割込み）。ディスクを読む装置が原因と思って「外部」を選ぶ誤りが多い。

割込み処理の間は、同じ種類の割込みを**禁止**（マスク）して、保存した状態が壊れないようにする。シラバスの「割込み禁止」はこのこと。

出典：IPA シラバス Ver.7.2 中分類5 1.(4)②の用語例（SVC割込み・入出力終了割込み・割込み禁止・ディスパッチ）／中分類3（割込みの種類）

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム ／ 応用情報.com 用語集 3-1 プロセッサ

ジョブ→ジョブステップ→タスクの順に細かくなる。一時ファイルは表で数える

ジョブ
利用者の仕事
▶
ジョブステップ
プログラム1本の実行
▶
タスク
CPUを割り当てる単位

ジョブ管理の部品	役目
ジョブスケジューラー	リーダ（読込み）→イニシエータ（開始）→ターミネータ（後始末）→ライタ（結果の出力）
マスタスケジューラー	オペレーターとのやり取りと、システム全体の監視
JCL	ジョブの中身をOSに指示する言語

夜間にまとめて流す**バッチ処理**、定期実行の cron、裏で動かす**バックグラウンドジョブ**も、ジョブの動かし方の言葉。

R4春 問16：多重度2、各ジョブ50Mバイトの一時ファイル

時間の枠	動くジョブ	残っているファイル	合計
①	A	a	50
②	B・C	a（Bが参照）・b・c	150
③	D・E	b（Dが参照）・c（Eが参照）・d・e	**200**
④	F	d・e（Fが参照）・f	150

ファイルは「作ったジョブの**直後のジョブ**が終わるまで」残る。Bが終わるとaが消え、DとEが終わるとbとcが消える。最大の③が答え（**200Mバイト**）。

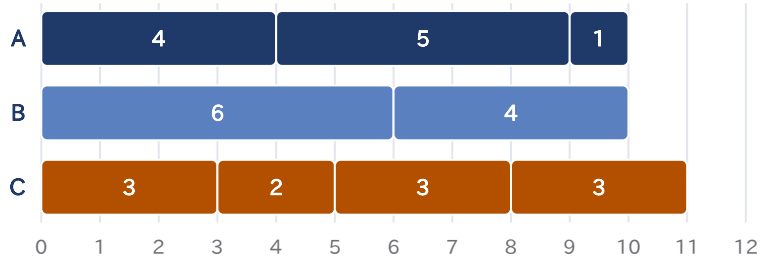
詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

遅い装置への出力はディスクにためる。割当ては空いた時刻の順に追う

プログラム
出力して次へ進む
▶
ディスク
スプール（待ち行列）
▶
プリンタ
空いたら少しずつ印刷

仕組み	ためる場所と単位
スプーリング	ディスクに、ジョブ（印刷データ）の単位でためる。CPUが装置を待たない
バッファリング	主記憶のバッファに、入出力の単位でためて速度差を吸収
DMA（#3）	CPUを通さずに装置と主記憶の間で転送する

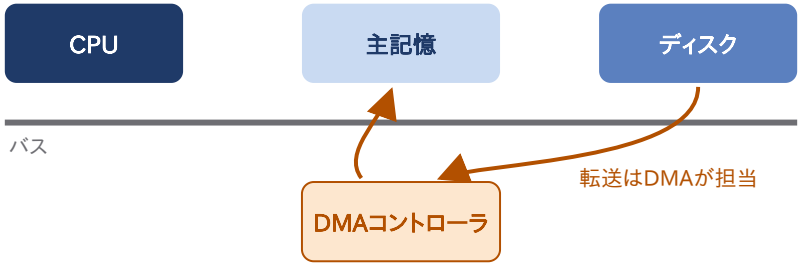
R4秋 問19：4,6,3,2,5,3,4,3,1 分を A→B→C の順に割当て（目盛りは分）



時刻0にA・B・Cへ。あとは**空いた時刻の早い順**に次のデータを渡す（3分にC、4分にA、5分にC、6分にB、8分にC、9分にA）。Cの合計は 3+2+3+3=**11分**。

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

入出力はCPUを通さずに送るほど速い。装置の番地の付け方は2通り



CPUは最初の指示と、終わりの入出力割込みを受けるだけ

方式	転送するもの
プログラム制御	CPUが1語ずつ読み書きする。CPUが占有される
DMA	専用の回路が主記憶と装置の間で直接送る
チャネル	入出力専用の小さなプロセッサ。汎用機で使う

入出力装置に番地を付ける2つの方式	メモリマップドI/O	I/Oマップド I/O
装置の番地	主記憶と**同じ**番地空間の一部	主記憶とは**別**の入出力ポート番号
使う命令	ふつうのロード・ストア命令	専用の入出力命令（x86 の IN・OUT）
長所	命令が少なくて済み、プログラムが簡単	主記憶の番地を装置に取られない

> IOCS（入出力制御システム）は、装置ごとの違いを隠してアプリに同じ呼出し方を見せる、OSの入出力の部分。

> スプーリング・バッファリング・DMA は、どれも「**CPUが遅い装置を待つ時間**を減らす」ための仕組み。

出典：IPA シラバス Ver.7.2 中分類5 1.（6）の用語例／Wikipedia「Memory-mapped I/O and port-mapped I/O」

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

使用率は「装置が動いていた時間÷全体の時間」。時間の線を引いて数える

H21春 問17：A・Bの順に実行。CPUの後にディスク。装置は1つずつ処理

ジョブ	CPU	ディスク
A	3秒	7秒
B	12秒	10秒



1. CPU：Aが0〜3秒、Bが3〜15秒
2. ディスク：Aは3秒から7秒で10秒まで。BはCPUが終わる15秒から10秒で**25秒**まで
3. CPUの使用率＝(3＋12)÷25＝**0.60**
4. ディスクの使用率＝(7＋10)÷25＝**0.68**（正解 イ）

全体の時間を「CPUの合計＋ディスクの合計＝32秒」としない。**AのディスクとBのCPUが重なる**（3〜10秒）ので、全体は25秒。この重なりがマルチプログラミングの効果。

出典：IPA 平成21年度春期 午前 問17（平成24年度秋期 問15も同じ問題。表とグラフを文字と図で書き直した）

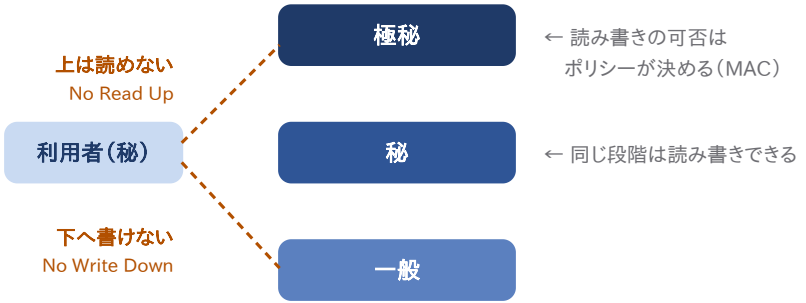
詳しく：応用情報.com 用語集 3-1 プロセッサ

起動は小さなプログラムからの順送り。アクセスは「誰が決めるか」で2種類



この順送りを**ブートストラップ**（IPL）と呼ぶ。サーバからOSを受け取るのが**ネットワークブート**、起動時にOSを選ぶのが**マルチブート**（同時に動くOSは1つ）。

アクセス制御	許可を決める人
任意（DAC）	ファイルの**所有者**が自分で決める。ふつうのOSのアクセス権
強制（MAC）	システムの**ポリシー**が決め、所有者でも変えられない。セキュアOS



BLPモデルは機密性を守る規則（上を読まない・下へ書かない）。利用者は**root**（**Administrator**）・一般ユーザー・ゲストに分け、ふだんは最小の権限で使う（WindowsのUAC）。アカウントの一元管理は**LDAP**・**Active Directory**。

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

普段は一般ユーザーで使い、必要な時だけ管理者の権限に上げる

アカウント	できること
スーパーユーザー（UNIXの root、Windowsの Administrator）	全部。他人のファイルも読め、OSの設定も変えられる
一般ユーザー	自分のファイルと、許された操作だけ
ゲスト	一時的な利用。最低限の操作だけ

UAC Windows のユーザーアカウント制御。管理者でログインしていても、普段は一般ユーザーの権限で動き、設定変更などの時だけ**確認の画面**を出して管理者の権限に上げる。UNIX の **sudo** と同じ考え方（最小権限）。

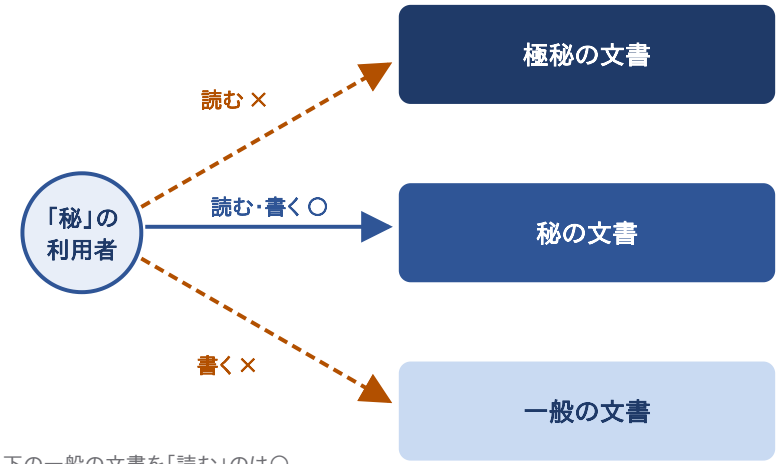
運用の機能	何をするか
ディスククオータ	利用者ごとにディスクの**使用量の上限**を決める
プロファイル	利用者ごとの環境設定（画面・パスなど）
システム利用権・ファイルアクセス権	ログインできるか・どのファイルに何ができるか

アカウントの一元管理	
LDAP	ディレクトリサービス（利用者・機器の名簿）を読み書きする**プロトコル**
Active Directory	Microsoft のディレクトリサービス**製品**。LDAP や Kerberos で、ドメイン内の利用者・PCをまとめて管理

出典：IPA シラバス Ver.7.2 中分類5 1.（9）（10）の用語例／Microsoft Learn「ユーザー アカウント制御」

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

BLPモデルは機密を守る規則。「上は読めない」「下には書けない」



下の一般の文書を「読む」のは○

BLPの規則	ねらい
上位を読まない（no read up）	自分より機密度が高い情報は見られない
下位に書かない（no write down）	機密の内容を、低い文書へ写して漏らさない

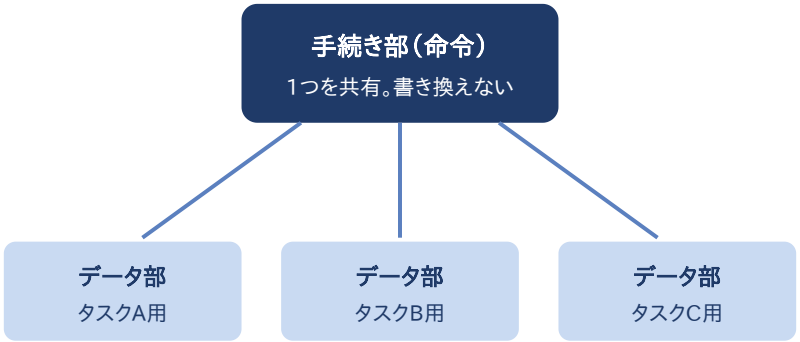
> 「書くのは上だけ」は不思議に見えるが、**漏えい防止**（機密性）が目的だから。逆に改ざんを防ぐ（完全性）のは Biba モデルで、規則が上下逆になる。

OSのセキュリティ制御	中身
多重レベルセキュリティ	情報と利用者に機密度のラベルを付けて管理（強制アクセス制御）
ロギング・オーディット	誰が何をしたか記録し、後から監査できる（アカウンタビリティ）
信頼プロセス	規則の例外を許された、検証済みのプロセス

出典：IPA シラバス Ver.7.2 中分類5 1.（11）の用語例／Wikipedia「Bell–LaPadula model」

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

同時に何人が呼んでも正しく動くのが再入可能。データをタスクごとに分ける



変数はスタックなどタスクごとの領域に置く

性質	意味	使われ方
再入可能（リエントラント）	実行中に別のタスクから呼ばれても正しく動く	複数のタスクが**同時に**共用する
逐次再使用可能	終われば次の人がロードし直さずに使える。同時には使えない	1人ずつ**順番に**使う
再帰的（リカーシブ）	自分自身を呼び出せる	木の探索・階乗
再配置可能	主記憶のどこに載せても動く	載せる場所が毎回変わる

再入可能 ⊂ 逐次再使用可能。再入可能にするには、命令の中に変数を置かず（**自分を書き換えない**）、作業用のデータをタスクごとに持たせる。再帰呼出しも同じ条件が要る。

出典：IPA シラバス Ver.7.2 中分類5 1.（13）の用語例（リエントラント・逐次再使用可能）／Wikipedia「Reentrancy (computing)」

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

⏳ 裏で常に動いて頼まれ事を待つのがデーモン。cronは5つの欄で時刻を決める

デーモンの例	役目
cron（crond）	決めた時刻にコマンドを動かす
sshd	リモートからのログインを受け付ける
httpd	Webのリクエストに応える（Apache）
印刷のデーモン（cupsd）	プリントキューから順に印刷する

過去問 H22秋 問20「UNIXのデーモン」の正解は「OSと同時又は必要に応じて起動され、**バックグラウンドで常に動作**し、プリントキューの印刷や通信の機能を提供する」。誤りの選択肢は、アプリ間の連携（OLEなど）、**API**（呼出し方式が同じならOSの差を意識しない）、**シェル**（ログインすると起動される利用者とのインタフェース）。

crontab の書き方：分 時 日 月 曜日 コマンド	
書いた行	**動く時刻**
`0 2 * * * backup.sh`	毎日 2時0分
`30 9 * * 1-5 report.sh`	平日（月〜金）の 9時30分
`*/10 * * * * check.sh`	10分ごと
`0 0 1 * * bill.sh`	毎月1日の 0時0分

「*」はその欄の全部。曜日は 0 と 7 が日曜。ジョブを夜間にまとめて流す**バッチ処理**の自動運転によく使う。

出典：IPA 平成22年度秋期 午前 問20／Linux man-pages「crontab(5)」／IPA シラバス Ver.7.2 中分類5 1.（3）の用語例

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

第2章

タスク管理

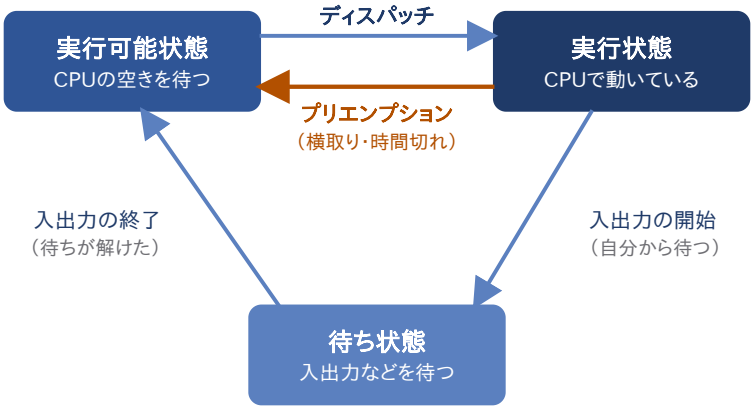
3つの状態 ／ プリエンプション ／ スケジューリング ／ ターンアラウンドタイム ／ 多重度 ／ 排他制御 ／ デッドロック






詳しく:応用情報.com 用語集 5-1 オペレーティングシステム

🚦 横取りされたタスクは「実行可能」に戻る。待ち状態ではない



待ち → 実行 には直接移らない（いったん実行可能へ）

状態	いつその状態か
実行状態	CPUを使って命令を実行している（CPU1個なら1つだけ）
実行可能状態	CPUさえあればすぐ動ける。待ち行列に並んでいる
待ち状態	入出力の終了・資源・イベントを待っていて、CPUが空いても動けない
休止状態（RTOS）	登録されているが起動前、または実行を終えた

R3春 問17・R7春 問14（同じ問題）：リアルタイムOSで、プリエンプションによって移る状態は**実行可能状態**。「待ち状態」「休止状態」「終了状態」は誤り。

詳しく：応用情報.com 用語集 3-1 プロセッサ

スレッドはプロセスの中で、メモリを共有して並行に動く「軽いプロセス」



	プロセスを増やす	スレッドを増やす
メモリ	別々。他を壊しにくい	**共有**。受け渡しが速い
作る・切り替える手間	大きい（空間ごと用意）	**小さい**（スタックとレジスタだけ）
注意点	データの受け渡しに通信が要る	同じ変数を同時に書くと**結果が狂う**→排他制御
例	ブラウザのタブごとのプロセス	Webサーバの1リクエスト1スレッド

> シラバスの**軽量プロセス**はスレッドのこと。CPUが複数あれば、同じプロセスのスレッドが本当に**同時**に動く（マルチスレッド）。どのスレッドから呼ばれても正しく動く関数を**スレッドセーフ**という。

出典：Linux man-pages「pthreads(7)」（スレッドはデータとヒープを共有し、スタックは各自が持つ）／IPA シラバス Ver.7.2 中分類5 1.（4）①の用語例

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

状態の問題は「誰がCPUを取るか」を1つ決めれば、残りは自動で決まる

H24春 問22：CPU1個・プリエンプティブ。優先度 A＞B＞C			
時点	**A**	**B**	**C**
今	待ち	実行	実行可能
Aの待ちが解消した後	**実行**	**実行可能**	実行可能

1. Aは待ちが解けて実行可能になり、**最も優先度が高い**ので CPU を取る→実行
2. Bは CPU を取り上げられる（プリエンプション）→**実行可能**（待ちではない）
3. Cは何も起きていない→実行可能のまま。正解は**エ**

過去問 H21秋 問19：タスクAが実行状態から**実行可能状態**へ移るのは「Aより優先度の高いタスクBが**実行状態**となった」とき（正解 ウ）。

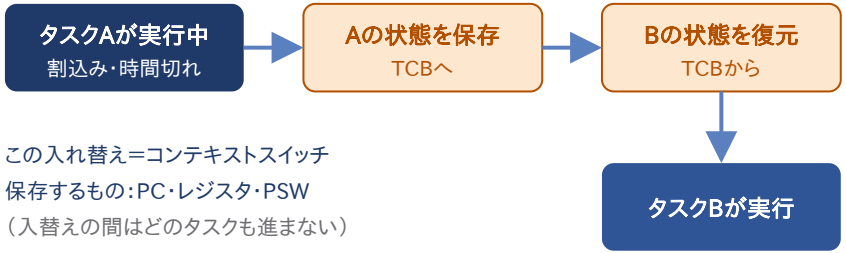
誤りの選択肢	Aはどうなるか
Aが入出力要求のシステムコールを出した	入出力の終わりを待つので**待ち状態**
Aが優先度の低いBにメッセージを送った	Aの方が高いので、Aは**実行のまま**
Aより高いBが**待ち状態**になった	Bは CPU を使わないので、Aは実行のまま

> 休止状態は「まだ起動されていない・終わった」タスク。横取りで休止状態になることはない。

出典：IPA 平成24年度春期 午前 問22（選択肢の表を文字で書き直した）・平成21年度秋期 午前 問19

詳しく：応用情報.com 用語集 3-1 プロセッサ

ディスパッチは「CPUを渡すこと」。順番を決めるスケジューリングとは別



この入れ替え＝コンテキストスイッチ
保存するもの：PC・レジスタ・PSW
（入替えの間はどのタスクも進まない）

タスクの情報を入れておく表を **TCB**（タスク制御ブロック、プロセスなら PCB）という。状態・優先度・保存したレジスタなどを持つ。

過去問 H23秋 問19「タスクのディスパッチ」の正解は「あるタスクの実行中に、別のタスクに**切り替え、かつ実行権を渡す**こと」。

誤りの選択肢	本当は何か
各タスクの実行順序を決定すること	スケジューリング
タスクの実行に必要な各種情報のこと	**コンテキスト**（TCBに入る中身）
複数のタスクを同時に実行しているかのように見せかけた状態	**マルチタスク**（多重プログラミング）

切替えが多すぎると、保存と復元の時間（オーバーヘッド）が増える。ラウンドロビンのタイムクウォンタムを小さくしすぎない理由。

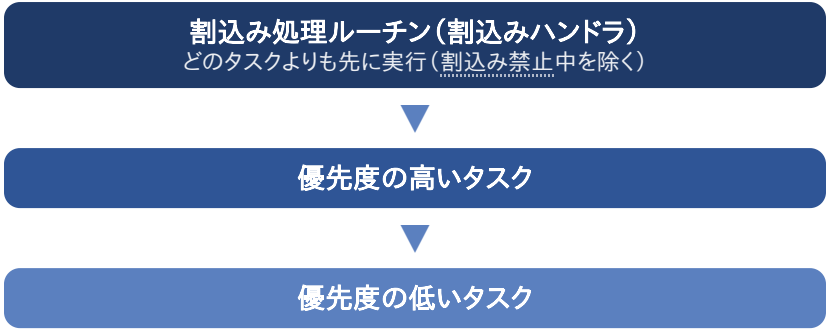
出典：IPA 平成23年度秋期 午前 問19／IPA シラバス Ver.7.2 中分類5 1.(4)①の用語例（ディスパッチャ）

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

🔔 OSが横取りできるのがプリエンプティブ。割込み処理はどのタスクよりも先

	プリエンプティブ	ノンプリエンプティブ
切り替わるとき	OSの判断で、いつでも（優先度の高いタスクの到着・時間切れ）	タスクが**自分から**手放したときだけ（終了・待ち・中断の要求）
無限ループのタスク	他のタスクは動ける	他のタスクが**動けなくなる**
使われる所	リアルタイムOS・今のPC用OS	昔のOS・単純な組込み機器

R6春 問16：ノンプリエンプティブは「自らの中断をOSに要求したときだけ切り替わる」。「優先度の高いタスクが来たら中断」はプリエンプティブの動き。



R4秋 問18：割込み処理ルーチンは**最も優先度の高いタスクよりも優先**して実行される。処理が長いと応答性が落ちるので短く作る。割り込まれたタスクとは**別のコンテキスト**で動き、タスク切替えを禁止しても割込みは止まらない。

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

🚦 横取りは「自分より優先度が高い」タスクが動けるようになった時だけ

H27秋 問16・H30秋 問17：優先度 A ＞ B ＝ C ＞ D。Bが実行中

起きたこと	Bの横取りは？
Aが実行可能状態になった	**起きる**。AはBより高い（正解 ア）
Bが待ち状態になった	Bが**自分から**CPUを手放しただけ。横取りではない
Cが実行可能状態になった	BとCは**同じ**優先度。後から来たCは待つ
Dが実行可能状態になった	DはBより低い。待つ



同じ優先度どうしは、到着順に並ぶ。同じ優先度の間でも時間で区切って交代させたいときは、**ラウンドロビン**を組み合わせる。

「プリエンプション」は横取りそのもの。自分から待ちに入るのは**横取りではない**点が狙われる。

出典：IPA 平成27年度秋期 午前 問16（平成30年度秋期 午前 問17も同じ問題）

詳しく：応用情報.com 用語集 3-1 プロセッサ

⏳ 短い処理を優先すると、長い処理がいつまでも待たされることがある

方式	次に動かすタスク	横取り	押さえどころ
到着順（FCFS）	先に来たもの	しない	公平で簡単。長い処理の後ろに並んだ短い処理が待たされる
処理時間順（SJF）	処理時間が最も短いもの	しない	平均の待ち時間は最短。短いものが次々来ると、長いものが**待ち続ける（スタベーション）**
優先度順	優先度が最も高いもの	することが多い	低い優先度が待たされる。待つほど優先度を上げる**エージング**で防ぐ
ラウンドロビン	待ち行列の先頭	する（時間切れで最後尾へ）	全員に順番が回る。タイムクウォンタムが短すぎると切替えの手間が増える
多段フィードバック待ち行列	高い段の待ち行列の先頭	する（時間切れで一段下へ）	短い処理は早く終わる。処理時間を知らなくてもSJFに近い効果

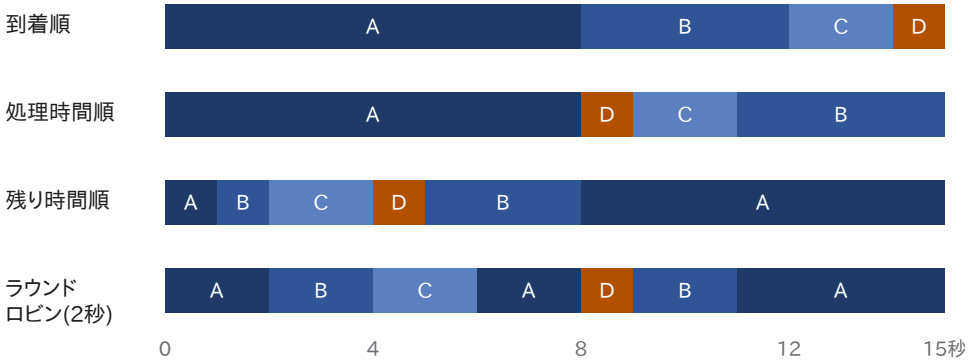
R6秋 問16：「特定のタスクがCPUの割当てを待ち続ける可能性が最も高い」のは**処理時間順（SJF）**。エージング付きの優先度順・ラウンドロビン・到着順は、待てばいつか順番が来る。



詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

⏱️ 同じジョブを4つの方式で回すと、平均のターンアラウンドがこれだけ変わる

ジョブ（到着・処理時間）：A(0秒・8) B(1秒・4) C(2秒・2) D(3秒・1)。CPU 1個



方式	A・B・C・D の終了−到着	平均
到着順（FCFS）	8・11・12・12	10.75秒
処理時間順（SJF）	8・14・9・6	9.25秒
残り時間順（横取りあり）	15・7・2・2	**6.5秒**
ラウンドロビン	15・10・4・6	8.75秒

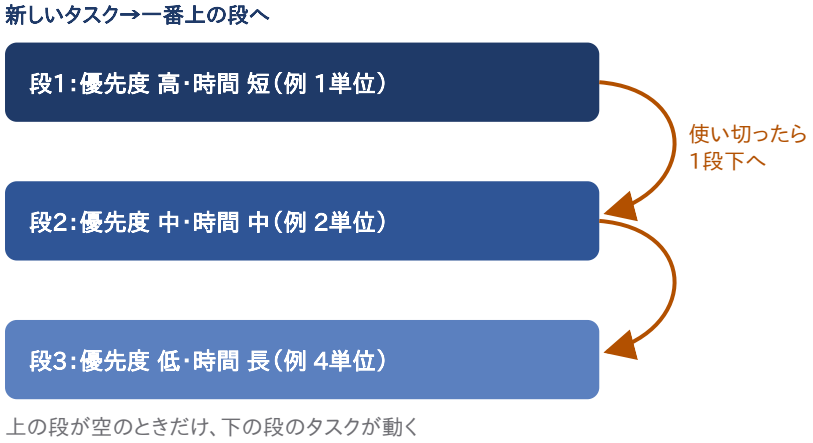
短いものを先にするほど平均は短くなる。その代わり長いA（15秒）は最後まで待たされる。これが**スタベーション**の芽。

ラウンドロビンでは「新しく来たジョブ」を「時間切れで戻るジョブ」より前に並べた。問題文の約束を必ず読む。

出典：Claude が作った例題（同じ条件で4方式を手で計算）／IPA シラバス Ver.7.2 中分類5 1.(4)②の用語例（FCFS・SJF・ラウンドロビン・処理時間順方式）

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

フィードバック待ち行列は、CPUを長く使うほど下の段へ落とす



方式	しくみ
フィードバック待ち行列	使った時間で段を下げる。対話的な短い処理が自然に優先される
静的優先順位	優先度を最初に決めて変えない
動的優先順位	待った時間などで優先度を変える。待つほど上げるのが**エージング**
タイムスライス	CPUを一定時間（タイムクウォンタム）ずつ区切って渡す
イベントドリブン	入出力の終了などの出来事（イベント）を契機に切り替える

> フィードバック待ち行列は1962年に Corbató が考案した。処理時間を**前もって知らなくても**、短い処理を優先した SJF に近い効果が出る。

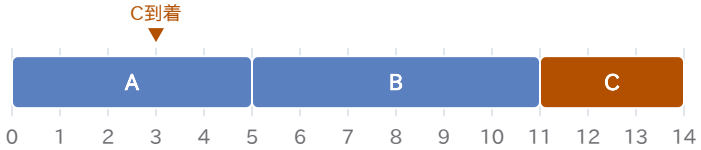
出典：Wikipedia「Multilevel feedback queue」／IPA シラバス Ver.7.2 中分類5 1.（4）②の用語例

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

⏱ ターンアラウンドタイムは「終了時刻ー到着時刻」。時間の線を書けば数えられる

R3春 問16：到着順・多重度1

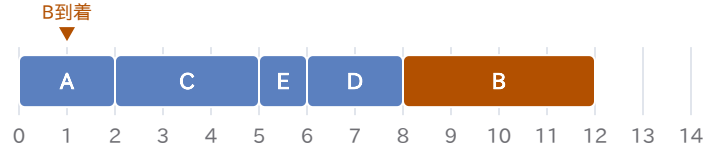
ジョブ	到着	処理
A	0	5
B	2	6
C	3	3



Cは11秒に始まり14秒に終わる。**14−3=11秒**（待った8秒＋実行3秒）。

R7秋 問15：処理時間順・多重度1

ジョブ	A	B	C	D	E
到着	0	1	2	3	4
処理	2	4	3	2	1



1つ終わるたびに、**その時点で到着済み**のものから最短を選ぶ。2秒：B・CからC。5秒：B・D・EからE→D→B。**12−1=11秒**。

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

ラウンドロビンは待ち行列の中身を時刻ごとに書けば迷わない

H28秋 問19：タイムクウォンタム2秒。同時なら新しいプロセスを先に並べる

プロセス	生成時刻	処理時間
1	0秒	5秒
2	3秒	7秒
3	6秒	5秒

時刻	実行	待ち行列（先頭←）	残り
0〜4	1（他に誰もいない）	3秒に2が並ぶ	1:1
4〜6	2	1	2:5
6	—	1・**3**・2（新しい3が2より前）	
6〜7	1 終了	3・2	1:0
7〜13	3・2・3 と交代		2:3→…
13〜17	2・3（16秒終了）・2		**2は17秒に終了**

1. 2は生成から17−3＝**14秒後**に終わる（正解 イ）
2. 6秒は「2の時間切れ」と「3の生成」が**同時**。約束どおり3を先に並べる
3. 1は0〜2秒、2〜4秒と**続けて**動く。並んでいる相手がいなければ交代しない

誤りの 12秒・16秒・17秒は、6秒の並べ方を逆にしたり、生成時刻を引き忘れたりすると出る数。

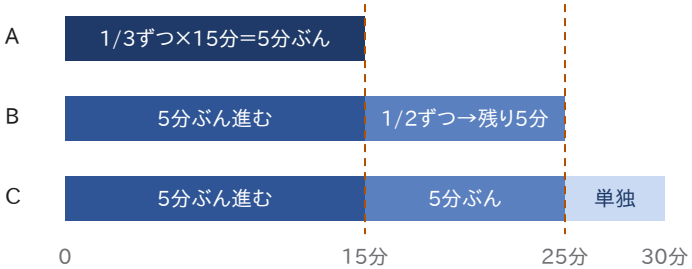
ラウンドロビンは**タイムシェアリング**（多くの利用者が1台を交代で使う）に向く（H26春 問17）。切替えにタイマーが要るので、タイマー機能のないシステムでは作れない。

出典：IPA 平成28年度秋期 午前 問19（表を文字で書き直した）・平成26年度春期 午前 問17（平成29年度秋期 問18も同じ問題）

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

⏳ タイムクウォンタムがとても小さいと、n本のジョブがCPUを n等分して進む

H30秋 問16：A 5分・B 10分・C 15分を同時に開始。Bが終わるのは？



3本で分ける間は、1本の進み方が実時間の1/3になる

1. 3本が並ぶ間、1本は1分あたり1/3分進む。一番短いA（5分）が終わるのは 5×3＝**15分**
2. この時点でBもCも5分ぶん進んだ。Bの残りは5分
3. 2本で分けるので 5×2＝10分かかり、Bは15＋10＝**25分**で終わる（正解 ウ）
4. 最後のCは単独で残り5分、**30分**で終わる。合計30分は処理時間の合計と一致する

> この「均等に分ける」見方は、タイムクウォンタムが**十分に小さい**と書かれたときだけ使える。2秒などの具体的な値なら、前のページのように表で追う。

出典：IPA 平成30年度秋期 午前 問16

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

多重度は「同時に動ける数」。要求の到着時刻をずらして線を引く

R4春 問19・R6春 問18（同じ問題）

多重度2以下なら1件4秒。1秒おきに4件届く。多重度1と2で、全部終わるまでの時間の差は？



多重度	考え方	終わる時刻
1	1件ずつ順に4秒	16秒
2	2本の列で並行。2件目は1秒に到着して1〜5秒、4件目は5秒に始まる	9秒

差は **16−9＝7秒**。「4件÷2×4秒＝8秒」と割り算すると、到着が1秒ずつずれることを見落とす。

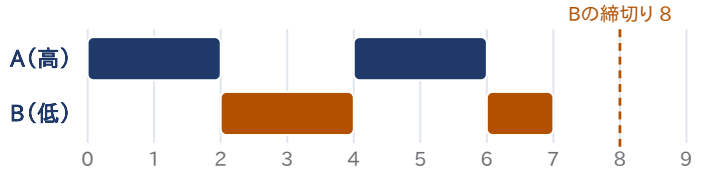
多重度を上げすぎると、主記憶が足りなくなりページの出し入ればかりになる（第3章のスラッシング）。

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

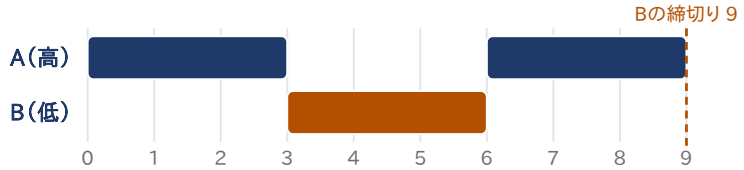
周期タスクは、高い方が使った残りの時間で低い方が終わるかを数える

R5秋 問17：Aの優先度が高い。Bは周期内に終わるか

ア A：2ミリ秒／周期4　B：3ミリ秒／周期8



イ A：3ミリ秒／周期6　B：4ミリ秒／周期9



選択肢	Bの周期内でAが使う時間	残り	Bの必要
ア	2+2=4	8−4=4	3 ○
イ	3+3=6	9−6=3	4 ×
ウ	3+3+3=9	13−9=4	5 ×
エ	4+4+3=11	15−11=4	5 ×

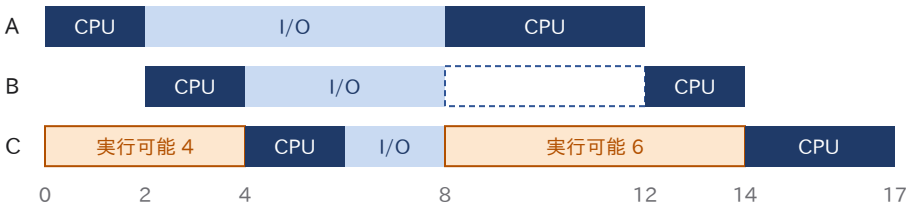
Aは周期が来るたびに**Bを横取りして**先に動く（プリエンプティブ）。エは3回目のAが12〜16ミリ秒にかかるが、Bの締切り15までの3ミリ秒を数える。

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

CPUとI/Oが交互のタスクは、優先度の高い方から1本ずつ線を引く

H30春 問17：I/Oは競合しない。Cが実行可能状態にある時間の合計は？

タスク	優先度	単独で実行したときの順序（ミリ秒）
A	高	CPU 2 → I/O 6 → CPU 4
B	中	CPU 2 → I/O 4 → CPU 2
C	低	CPU 2 → I/O 2 → CPU 3



1. 0〜2 A、2〜4 B、4〜6 C が CPU。Cは0〜4の間、実行可能で待つ（**4ミリ秒**）
2. 8ミリ秒で3本のI/Oがそろって終わる。A（8〜12）→B（12〜14）の順に動く
3. Cは8〜14の間、また実行可能で待つ（**6ミリ秒**）。14〜17に動いて終わる
4. 合計 4+6=**10ミリ秒**（正解 ウ）。I/O中の時間は「待ち状態」なので数えない

> 「実行可能」は CPU を待つ時間だけ。I/Oの最中は**待ち状態**。2つを混ぜると 12 になる。

出典：IPA 平成30年度春期 午前 問17（表を文字と図で書き直した）

詳しく：応用情報.com 用語集 3-1 プロセッサ

I/Oが1台しかないときは、CPUだけでなくI/Oの順番待ちも線に書く

H24春 問20：CPU 1個・I/Oも同時に1つだけ。全部終わるのは何ミリ秒後？

優先度	単独で実行したときの順序（ミリ秒）
高	CPU 2 → I/O 7 → CPU 3 → I/O 4 → CPU 3
低	CPU 2 → I/O 3 → CPU 2 → I/O 2 → CPU 3



1. 低は2〜4にCPUを終えるが、I/Oは高が9まで使用中。低のI/Oは**9〜12**
2. 高は9〜12にCPU、12〜16にI/O。低は12〜14にCPU、I/Oは16まで待って**16〜18**
3. 高は16〜19にCPUで終了。低は19〜22にCPUで終了。全体で**22ミリ秒**（正解 エ）

> I/Oの順番待ちを忘れると、低のI/Oを4〜7に置いてしまい19〜20 の誤りの選択肢になる。

出典：IPA 平成24年度春期 午前 問20（表とグラフを文字と図で書き直した）

詳しく：応用情報.com 用語集 3-1 プロセッサ

CPUを遊ばせない相手は「高のI/Oの間にちょうど収まる」CPUを持つタスク

H22秋 問18：高のタスクAと組み合わせて、CPUの遊休時間を0にできるのは？

タスク	単独で実行したときの順序（ミリ秒）
A（高）	CPU 3 → I/O 3 → CPU 3 → I/O 3 → CPU 2
ア B	CPU 2 → I/O 5 → CPU 2 → I/O 2 → CPU 3
イ C	CPU 3 → I/O 2 → CPU 3 → I/O 3 → CPU 2
ウ D	CPU 3 → I/O 2 → CPU 3 → I/O 1 → CPU 4
エ E	CPU 3 → I/O 4 → CPU 2 → I/O 5 → CPU 2



紺・青＝CPU、薄い色＝I/O。目盛りは1ミリ秒＝23px、0〜18ミリ秒

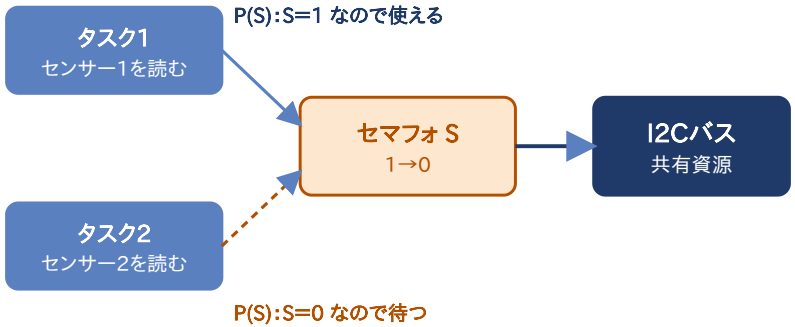
1. AのCPUは 0〜3・6〜9・12〜14。**3〜6 と 9〜12 が空く**（各3ミリ秒）
2. Dは3〜6にCPU、6〜8にI/O、9〜12にCPU（9まではAが使う）、12〜13にI/O
3. DのCPU 4はAの終わる14から18まで。0〜18の間、CPUは一度も空かない（正解 **ウ**）
4. Bは最初のCPUが2ミリ秒しかなく、5〜6が空く。Eも4ミリ秒のI/Oで9〜10が空く

> H28秋 問17 は同じ表で「遊休時間が最も短い組合せ」を問い、正解はやはり CPU 3→I/O 2→CPU 3→I/O 1→CPU 4 のタスク。

出典：IPA 平成22年度秋期 午前 問18・平成28年度秋期 午前 問17（表を文字で書き直した）

詳しく：応用情報.com 用語集 3-1 プロセッサ

共有資源を1つずつ使わせるのがセマフォ。キューやラウンドロビンは別の役



P操作：Sが1以上なら1減らして進む。0なら待つ
V操作：Sを1増やし、待っているタスクを起こす

R4春 問17・R7秋 問16 の選択肢	
セマフォ	**正解**。資源の空きを数で表し、P操作・V操作で確保と解放
キュー	タスク間でデータを順に受け渡す（メッセージキュー）
マルチスレッド	処理を並行して動かすこと。同時に**使わせない**仕組みではない
ラウンドロビン	CPUを順番に回すスケジューリングの方式
メモリプール	決まった大きさのメモリを貸し出す記憶管理（R7秋で入れ替わった選択肢）

> ロックしたタスクだけが解放できる**ミューテックス**、ループで待つ**スピンロック**も排他制御の道具。守る部分を**クリティカルセクション**と呼ぶ。

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

排他制御がないと、2人が同時に入金しただけで残高が狂う

残高1,000円の口座に、XとYが同時に入金する			
時刻	X（300円入金）	Y（500円入金）	口座の残高
1	残高を読む：1,000		1,000
2		残高を読む：1,000	1,000
3	1,000＋300 を書く		1,300
4		1,000＋500 を書く	1,500

正しくは1,800円。Xの書いた結果が、古い値をもとにしたYの書込みで**消える**（ロストアップデート）。タイミングで結果が変わる**競合状態**の典型。

セマフォで「読む〜書く」を1人ずつにする

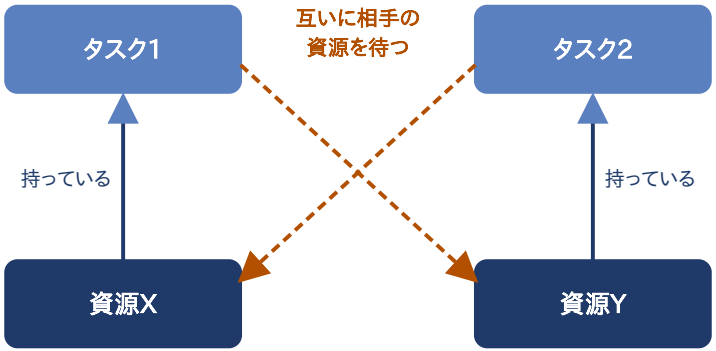
1. X：**P操作**（セマフォ 1→0）。入ってよい
2. Y：P操作。セマフォが0なので**待ち状態**になる
3. X：読む 1,000 → 書く 1,300 → **V操作**（Yを起こす）
4. Y：読む 1,300 → 書く **1,800** → V操作（0→1）

P操作〜V操作で囲んだ部分を**危険区域**（クリティカルセクション）という。値が0と1だけのものを2値セマフォ、プリンタ3台のように数を数えるものを計数セマフォという。

出典：Claude が作った例題／Wikipedia「Semaphore (programming)」（P・V操作、Dijkstra）／IPA 平成30年度春期 午前 問18（セマフォの目的は共有資源の管理）

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

デッドロックは待ちの輪。全員が同じ順番で資源を取れば輪ができない



取る順番が「X→Y」と「Y→X」で逆だと、待ちの輪ができる

R4秋 問16・R7秋 問18（同じ問題）：防ぐ方法は「資源獲得の順序を両方のタスクで**同じ**にする」。優先度を変えても防げない。

R6春 問17：Aと同時に動かしてデッドロックが起きうるのは？

プロセス	取る順（表の数字を並べ直す）	Aと
A	X → Y → Z	—
B	X → Y → Z	同じ順 ○
C	Z → X → Y	Xを持つAとZを持つCが**待ち合う**
D	Z → Y → X	XとYを逆に取り合う

答えは**C, D**。表の数字は「何番目に取るか」なので、まず資源を**取る順に並べ直す**のがこつ。



詳しく：応用情報.com 用語集 9-1 データベース方式

デッドロックは4つの条件がそろうと起きる。1つ崩せば防げる

条件	意味	崩し方
相互排除	資源を1度に1人しか使えない	共有できる形にする（印刷はスプールで受け付けるなど）
保持と待ち	持ったまま、別の資源を待つ	必要な資源を**最初に全部**まとめて取る
横取りできない	持っている資源を取り上げられない	待つことになったら、持っている資源を**一度返す**
循環待ち	A→B→…→A と待ちが輪になる	資源に番号を付け、**全員が同じ順**で取る

> 過去問の正解はほぼ「循環待ち」を崩す方法（取る順をそろえる）。H31春 問18・R4秋 問16・R7秋 問18 が同じ考え方。

見分け方 表で「資源を取る順」が並ぶ問題は、**2つのプロセスの順番が逆になっている資源の組**を探す。
例：Aは X→Y、Bは Y→X なら、AがXを、BがYを持った瞬間に輪ができる。

デッドロック以外の対処	考え方
回避（銀行家のアルゴリズム）	資源を渡す前に、全員が最後まで終われるかを計算して確かめる
検出と回復	待ちの輪を見つけたら、どれかを止めて資源を返させる（DBMS のデッドロック検出）

出典：Wikipedia「Deadlock (computer science)」（Coffman らが1971年に示した4条件）／IPA 平成31年度春期 午前 問18

詳しく：応用情報.com 用語集 9-1 データベース方式

並行処理の困りごとは5つ。「止まる」「進まない」「結果が狂う」で分ける

現象	何が起きるか
デッドロック	互いに待って**全員が止まる**
ライブロック	譲り合いを繰り返し、動いているのに**進まない**
スタベーション	他は動くのに、**特定のタスクだけ**待たされ続ける
優先度逆転	低い優先度のタスクがロックを持つ間に、中くらいが動き続け、高いタスクが待たされる。**優先度継承**で防ぐ
競合状態	同時に読み書きして、タイミングで**結果が狂う**

競合状態の例：残高100に、2つのスレッドが10ずつ足す



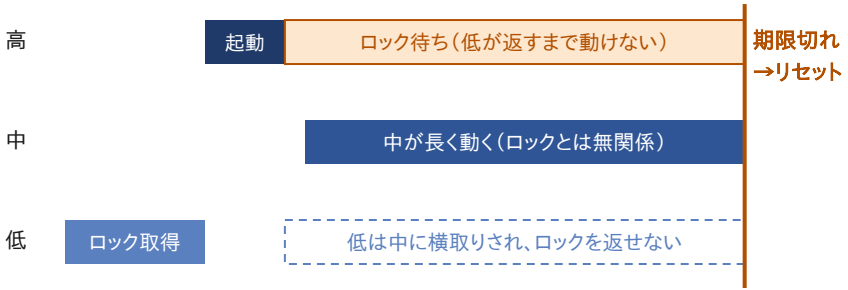
正しくは120。片方の更新が消える

読む→足す→書く をクリティカルセクションとして排他制御する

R7秋 問17：スレッドセーフは「複数のスレッドで並列に実行しても問題が生じない」性質。「複数のスレッドから呼ばれないようにする」「単一スレッドで動かす」は誤り。

詳しく：応用情報.com 用語集 9-1 データベース方式

優先度逆転は火星でも起きた。1997年のマーズ・パスファインダー



対策：優先度継承＝ロックを持つ低を、待っている高と同じ優先度に一時的に上げる

何が起きたか	事実
症状	着陸後、探査機のコンピュータが**リセット**を繰り返した（7月5・10・11・14日の4回）
原因	**優先度逆転**。高い優先度のタスクが、低いタスクの持つ資源を待つ間に、中くらいのタスクに CPU を取られ続けた
調べ方	飛行ソフトに残してあったログとデバッグ機能を使い、地上の同型機で再現した
直し方	7月21日、ソフトを修正して**優先度継承**を有効にした

> 止まった後はリセット（**ウォッチドッグタイマー**）で立ち直る作りだったので、データは失われなかった。障害管理の「タイマー監視」の実例でもある。

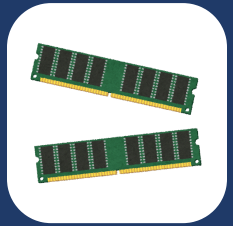
出典：Wikipedia「Mars Pathfinder」（リセットの日付・7月21日の修正）／Wikipedia「Priority inversion」

詳しく：応用情報.com 用語集 3-1 プロセッサ

第3章

記憶管理

区画方式 ／ 断片化とGC ／ 仮想記憶 ／ ページングとセグメンテーション ／ ページフォールト ／ ページ置換え（FIFO・LRU）／ スラッシング









詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

空きの合計は足りても、続いた空きがないと入らない。それが断片化



可変区画：ジョブごとに必要な大きさだけ

新しいジョブ 30

空きは 25・15・40
合計80あっても
入るのは 40 の所だけ

どの空きに入れるか

ファーストフィット：最初に見つかった入る空き

ベストフィット：入る中で一番小さい空き

ワーストフィット：一番大きい空き

空きが細切れになるのが断片化

→ 詰め直す（コンパクション）
連続した大きな空きを作る

方式	しくみ	弱み
固定区画	主記憶を決まった大きさに区切る	区画の中に余りが出る（**内部断片化**）
可変区画	必要な大きさだけ切り出す	区画の間に細かい空きが出る（**外部断片化**）
オーバーレイ	プログラムを分け、必要な部分だけ順に読み込む	分け方を作る人が決める
スワッピング	実行中でないプログラムを丸ごとディスクへ出す	出し入れの時間がかかる

R4春 問18：断片化は「空き領域の合計は十分なのに、必要な大きさの**連続した領域**を獲得できない」状態。R3春 問18：使われなくなったヒープ領域を回収するのは**ガーベジコレクション**（コンパクションや動的再配置と入れ替えた選択肢に注意）。

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

空き領域の選び方は3つ。同じ要求でも、入るか入らないかが変わる

空き領域（番地の順）：100K・500K・200K・300K・600K。要求：210K → 420K → 110K → 430K

要求	ファーストフィット（先頭から最初に入る所）	ベストフィット（入る中で一番小さい所）	ワーストフィット（一番大きい所）
210K	500K に入れる（残り290）	300K（残り90）	600K（残り390）
420K	600K（残り180）	500K（残り80）	500K（残り80）
110K	残りの290K（残り180）	200K（残り90）	残りの390K（残り280）
430K	入らない（最大300K）	**600K に入る**	入らない（最大300K）

この例ではベストフィットだけが全部入る。ただし、いつもそうではない。ベストフィットは**使えない小さな切れ端**（80K・90Kなど）を多く残しやすい。

方式	長所	短所
ファーストフィット	探す時間が短い	先頭付近が細かく割れる
ベストフィット	大きな空きを残せる	全部を探す。小さな切れ端が増える
ワーストフィット	残りが大きく、次に使いやすい	大きな空きがすぐなくなる

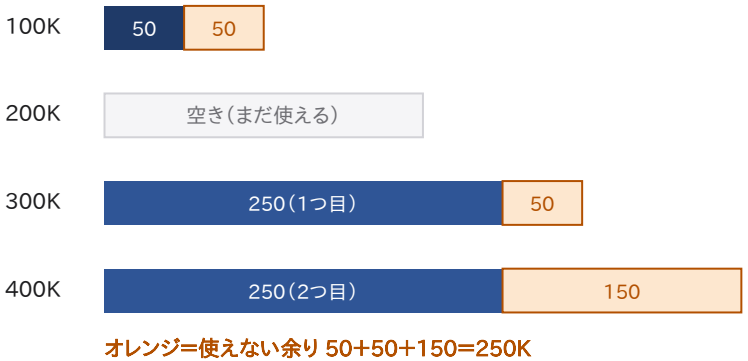
対策 散らばった空きを寄せ集めて1つにまとめるのが**メモリコンパクション**。動かす間はプログラムを止める必要があり、番地も付け直す（再配置）。

出典：Claude が作った例題／IPA シラバス Ver.7.2 中分類5 1.（7）①の用語例（ファーストフィット・ベストフィット・ワーストフィット・メモリコンパクション）

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

固定区画では「区画の中の余り」が使えない領域になる

H26秋 問17：区画 100K・200K・300K・400K。250K・250K・50K をベストフィットで割当て



1. 1つ目の250K：入る区画は300K・400K。小さい方の**300K**（余り50）
2. 2つ目の250K：残りで入るのは**400K**だけ（余り150）
3. 50K：入る中で一番小さい**100K**（余り50）
4. 余りの合計 50+150+50=**250K**（正解 イ）

200Kの区画は空いたまま。**まだ使える**ので「使えない領域」に入れない。入れると450K（エ）の誤りになる。

区画の**中**にできる余り＝内部断片化。可変区画で区画の**間**にできる細かい空き＝外部断片化。

出典：IPA 平成26年度秋期 午前 問17

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

オーバーレイは必要な部品だけ順に読み込む。空きの「位置」まで追う

H27春 問18：500Kの連続領域。ファーストフィット。最後の120Kは入るか

順序A：100→200→200解放→150→100解放→80→100→120　／　順序B：200→100→150→100解放→80→200解放→100→120

順序A（120を読む直前）



順序B（120を読む直前）



Bは空きの合計 100+20+50=170K あるのに、続いた120Kがない→入らない

0K　　500K

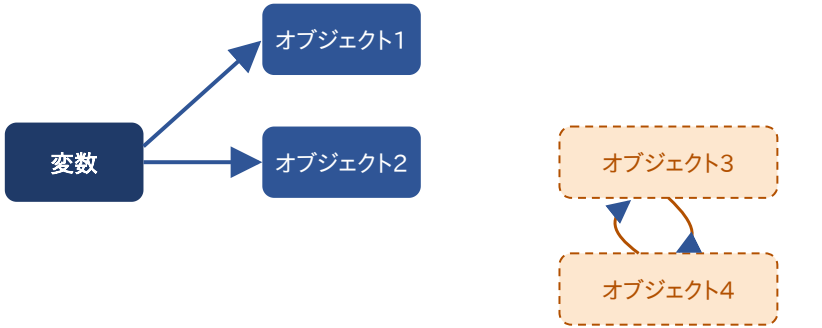
1. A：100(0〜100)・200(100〜300)→200を解放→150は先頭の空き(100〜250)へ
2. A：100を解放→80は0〜80、100は250〜350。残りは350〜500の**150K**で120が入る
3. B：200・100・150 を順に置き、100を解放→80は200〜280へ
4. B：200を解放→100は0〜100。空きは100K・20K・50Kに割れて**入らない**（正解 イ）

> 外部断片化の具体例。空きの合計ではなく、**続いた一番大きな空き**で判断する。

出典：IPA 平成27年度春期 午前 問18（読込み順序の図を文字と図で書き直した）

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

♻ 誰からも指されなくなった領域を回収するのがガーベジコレクション



言葉	意味
ヒープ	実行中に動的に確保・解放する領域（誤りの選択肢）
スタック	関数の呼出しごとに積み、戻ると下ろす領域（誤りの選択肢）
フラグメンテーション	空きが細かく散らばる**状態**。回収する機能ではない
メモリリーク	解放し忘れた領域がたまり、使える量が減っていく**不具合**

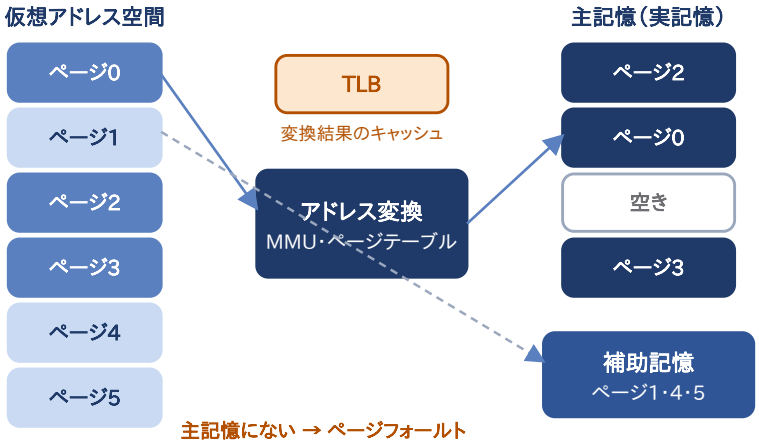
過去問 R2秋 問16（H25秋 問18も同じ）：「動的な割当てと解放を繰り返して発生した、**どこからも利用されない領域**を再び使えるようにする機能」はガーベジコレクション。

実例：Python は参照の数を数えて0になったら解放し（参照カウント）、それで回収できない**指し合いの輪**を gc モジュールの収集器が見つけて回収する。Java も実行環境が自動で回収する。C 言語には GC がなく、free を忘れるとメモリリークになる。

出典：IPA 令和2年度10月 午前 問16・平成25年度秋期 午前 問18／Python ドキュメント「gc — Garbage Collector interface」

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

仮想記憶は、主記憶より大きなプログラムを、必要なページだけ載せて動かす



プログラムは連続した広い空間に見える。実際は飛び飛びでも構わない

部品	役目
ページテーブル	仮想ページ番号→主記憶の枠番号の対応表。主記憶にあるかどうかの印も持つ
MMU	アドレス変換を行うハードウェア
TLB	最近の変換結果を覚えておく小さな高速メモリ。ページテーブルを毎回読まずに済む
補助記憶（ページング領域）	ページアウトされたページの置き場所

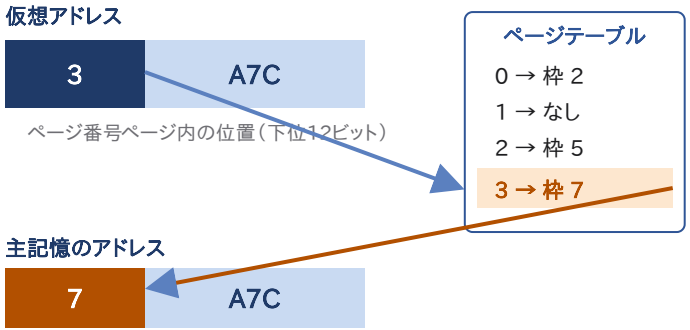
R3秋 問19：仮想記憶での補助記憶の役目は「**ページアウトされたページを格納する**」こと。更新前の内容の保存やバックアップではない。

ページは**必要になったときに読み込む**（デマンドページング）のがふつう。先に読んでおくのはプリページング。

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

アドレス変換は「ページ番号を枠番号に取り替え、ページ内の位置はそのまま」

例題：ページの大きさ 4Kバイト（16進で 1000）。仮想アドレス 3A7C は主記憶のどこ？



1. 4K＝2の12乗なので、下位12ビット（16進3桁）がページ内の位置。3A7C は**ページ3**の位置 **A7C**
2. ページテーブルで3を引くと**枠7**
3. 主記憶のアドレス＝7A7C（10進で 7×4096+2684＝31,356）
4. ページ1のように「なし」なら主記憶にない。**ページフォールト**が起きる

過去問 H26春 問18：論理アドレスから物理アドレスへの変換を行うのは「**主記憶に存在するページをアクセスするとき**」。フォールト・ページイン・ページアウトのときではない。

出典：Claude が作った例題／IPA 平成26年度春期 午前 問18／シラバス Ver.7.2 中分類5 1.（7）②の用語例（DAT・TLB）

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

実物のLinuxは4Kバイトのページと、何段にも重ねたページテーブルを使う

Linux カーネルの文書から	中身
ページの大きさ	ふつうは **4KB**。アドレスの下位12ビットがページ内の位置
最初の Linux	4KBのページと、1,024項目の表が1枚だけ。4MBぶん（作者の最初のPCの主記憶と同じ）
今の Linux	PGD → P4D → PUD → PMD → PTE の**最大5段**。大きなメモリを、使う所だけ表を作って管理する
ヒュージページ	2MB などの大きなページ。表を引く回数と TLB の消費を減らす

> 段が多いほど、TLB に無いときに表を何度も読む（**ページウォーク**）。だから TLB の当たり（ヒット）が性能を左右する。



過去問 H24秋 問18：ページングで断片化の問題を解決できる理由は「一連のプログラムやデータを、**不連続な主記憶**に割り付けることができる」から。ページ単位でばらばらの枠に置けるので、続いた大きな空きが要らない。

誤りの選択肢	なぜ違うか
仮想記憶のページ数を主記憶より多くできる	正しいが、大きさの話で断片化とは無関係
プログラム全体を1ページに割り付ける	ページは固定長。全体を1ページにはしない

出典：Linux Kernel Documentation「Page Tables」／Wikipedia「Translation lookaside buffer」／IPA 平成24年度秋期 午前 問18

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

📜 ページは固定長、セグメントは意味の単位。ページングは外部断片化なし

	ページング	セグメンテーション
分け方	**固定長**のページ	プログラム・データなど**論理的なまとまり**（可変長）
断片化	外部断片化は**起きない**（最後のページに内部断片化）	外部断片化が起きる
保護・共有	ページ単位なので意味と合わない	まとまりごとに保護や共用を決めやすい
大きさの変更	ページを足して対応	セグメントの長さを実行中に変えやすい

R6秋 問18：ページングの長所は「**外部断片化が発生しない**」。論理単位での保護・共用・大きさの動的変更は、どれも**セグメンテーション**の長所。

R4秋 問17：ページサイズを半分にすると

大半のプログラムがページの半分以下



1ページの中の使わない部分（内部断片化）が減る

→ 主記憶にたくさん載り、主記憶の不足が和らぐ（正解ウ）

逆に、ページを小さくするとページテーブルは大きくなる。「ページフォールトが増えて遅くなる」「余裕がなくなる」は、この問題の前提では誤り。

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

フォールト=ページイン ≧ ページアウト。平均時間は割合で重み付け

ページフォールト
ページが主記憶にない
▶
ページアウト
空きがなければ追い出す
▶
ページイン
補助記憶から読む

R3春 問19：フォールトが起きたら必ずページインするので**フォールト=ページイン**。ページアウトは空きがない時だけ（追い出すページが更新されていない時は書き戻しも不要）なので**それ以下**。答えは「PF=PI ≧ PO」。

ページアウトの対象をどれにするかが、次のスライドのページ置換えアルゴリズム。

R5秋 問16：平均処理時間30ミリ秒。ページアウトなしの割合は？

$$20 \times p + 60 \times (1 - p) = 30$$
$$60 - 40p = 30 \quad \rightarrow \quad p = 0.75$$

ページアウトなし20ミリ秒、あり60ミリ秒

R3秋 問17：許されるページフォールトの発生率

$$1\text{命令の遅れ} = 2\text{回} \times \text{率} \times 40\text{ミリ秒} \leqq 0.4\text{マイクロ秒}$$
$$\text{率} \leqq 0.4 \times 10^{-6} \div (80 \times 10^{-3}) = 5 \times 10^{-6}$$

ミリ（10^{-3}）とマイクロ（10^{-6}）の桁をそろえてから割る。「1命令で2回参照」の2を落とすと1×10^{-5}（イ）になる。

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

空きがないときのフォールト処理は「決める→追い出す→読み込む」の順

1. **ページフォールト**：主記憶にないページを参照した（プログラム割込み）
2. **置換え対象ページの決定**：FIFO・LRU などで、追い出すページを選ぶ
3. **ページアウト**：選んだページが更新されていれば補助記憶へ書き戻す
4. **ページイン**：空いた枠に、必要なページを補助記憶から読み込む
5. ページテーブルを書き換え、止まっていた命令を**やり直す**

過去問 R1秋 問18：空きページがないときの順番は「ページフォールト→置換え対象ページの決定→ページアウト→ページイン」（正解 ウ）。

誤りの選択肢の順番	おかしい所
決定→ページイン→フォールト→ページアウト	フォールトが起きる前に、何を決めるのか分からない
決定→フォールト→ページアウト→ページイン	フォールトが最初。きっかけが後ろにある
フォールト→決定→ページイン→ページアウト	枠が空く前に読み込むことはできない

追い出すページが**更新されていなければ**、補助記憶に同じ内容が残っているので書き戻し（ページアウト）は要らない。だから「ページアウトの回数 ≦ ページインの回数」になる。

出典：IPA 令和元年度秋期 午前 問18

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

⏱️ フォールト1回は主記憶アクセス何十万回分。1秒に何回まで耐えるか

H21春 問19：1回のアクセス200ナノ秒、フォールト1回の処理100ミリ秒、50万回に1回フォールト

1. アクセス50万回にかかる時間：500,000×200ナノ秒＝**0.1秒**
2. その間にフォールトが1回：処理に**0.1秒**
3. 「50万回のアクセス＋フォールト1回」の1組が0.2秒。1秒に**5組**
4. フォールトは1秒あたり最大**5回**（正解 ウ）

> フォールト処理の時間を足し忘れて「1秒÷0.1秒＝10回」としない。選択肢にない数になったら、足し忘れを疑う。

H25春 問18：1命令あたりの平均の遅れを式で表す

遅れ＝ t × f × m

記号	意味
m	1命令で主記憶を参照する平均回数
f	参照1回あたりのフォールトの起きる率
t	フォールト1回の処理時間

> 1命令あたり m 回参照 → フォールトは m×f 回 → 遅れは m×f×t。**単位を追えば**掛け算か割り算かが決まる（回×率×時間＝時間）。

出典：IPA 平成21年度春期 午前 問19・平成25年度春期 午前 問18

詳しく：応用情報.com 用語集 4-1 システムの構成

FIFOは「一番先に読み込んだページ」を出す。途中で使ったかは関係ない

R5春 問17：3つの枠、参照順 1, 4, 2, 4, 1, 3（枠の位置は動かない）

参照	枠1	枠2	枠3	結果
1	1			読込み
4	1	4		読込み
2	1	4	2	読込み
4	1	4	2	あり
1	1	4	2	あり
3	3	4	2	置換え（1を出す）

3を読む時、読み込んだ時刻が一番古いのは**ページ1**（直前に使ったばかりでも出す）。枠1が3になり、最後は 3, 4, 2（ウ）。

方式	追い出すページ	見る情報
FIFO	主記憶に**最も早く読み込んだ**ページ	読み込んだ時刻
LRU	**最後に使ってから最も時間がたった**ページ	最後に使った時刻
LFU	**使った回数**が最も少ないページ	使った回数
LIFO	最も新しく読み込んだページ	読み込んだ時刻
OPT（最適）	この先**最も長く使わない**ページ	未来の参照（実際には作れない比較の基準）

FIFOでは、枠の数を増やしたのにフォールトが増えることがある（**ビレディの逆説**）。

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

⏳ LRUは「最後に使った時刻」が一番古いページを出す。使うたびに時刻を更新

R7春 問15：4つの枠（4000〜7000番地）

参照順 1, 2, 3, 4, 2, 5, 3, 1, 6, 5, **4**。最後の4はどこに入る？

参照	4000	5000	6000	7000	結果
1〜4	1	2	3	4	読込み
2	1	**2**	3	4	あり
5	**5**	2	3	4	**置換え（1）**
3	5	2	**3**	4	あり
1	5	2	3	**1**	**置換え（4）**
6	5	**6**	3	1	**置換え（2）**
5	**5**	6	3	1	あり
4	5	6	**4**	1	**置換え（3）**

参照	その時の「最後に使った順」（古い→新しい）	出す
5	1, 3, 4, 2	1
1	4, 2, 5, 3	4
6	2, 5, 3, 1	2
4	3, 1, 6, 5	**3**

最後の4はページ3がいた**6000番地**（ウ）。「2」「3」「5」を使い直すたびに、その順番が一番新しい側へ動くのがFIFOとの違い。

R5春 問18：LRUは「**使用されてからの経過時間が最も長い**ページを置き換える」。使用回数で選ぶのはLFU。

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

同じ参照列でもFIFOとLRUで回数が違う。FIFOは枠を増やして悪化も

H23特別 問21：枠3、参照 1, 3, 2, 1, 4, 5, 2, 3, 4, 5。置換えの回数は？

参照	1	3	2	1	4	5	2	3	4	5
FIFO	1	1,3	1,3,2	あり	4,3,2	4,5,2	あり	4,5,3	あり	あり
LRU	1	1,3	1,3,2	あり	1,4,2	1,4,5	2,4,5	2,3,5	2,3,4	5,3,4

FIFO は**3回**、LRU は**6回**（正解 イ）。4番目に1を参照したので、LRU では1が「新しい」扱いになり、代わりに3や2が先に追い出される。この参照列では、それが裏目に出る。

LRU がいつも少ないとは限らない。問題ごとに表を書く。

ビレイディ（Bélády）の異常：FIFOで枠を増やしたのに増える

参照 3, 2, 1, 0, 3, 2, 4, 3, 2, 1, 0, 4

枠の数	FIFOのページフォールト
3	**9回**
4	**10回**（増える）

LRU では起きない。LRU は「枠がn個のときに入っているページは、n＋1個のときにも入っている」性質を持つため。FIFO は読み込んだ時刻だけを見るので、この性質がない。

出典：IPA 平成23年度特別 午前 問21（解答群の表を文字で書き直した）／Wikipedia「Bélády's anomaly」（参照列と9回・10回をClaudeが手で確かめた）

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

「フォールトの回数」と「ページアウトの回数」は数える物が違う

H24秋 問19：LRU・枠4。全部「参照かつ更新」。ページアウトは何回？

参照 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 6, 5

参照	枠の中身	追い出し
1〜4	1,2,3,4	空き枠に入る（4回フォールト）
5	1,2,**5**,4	3（最後の使用が一番古い）
3	1,2,5,**3**	4
6	1,2,**6**,3	5
5	**5**,2,6,3	1

フォールトは8回だが、最初の4回は空き枠に入るだけ。ページアウトは**4回**（正解 イ）。全ページが更新されているので、追い出し＝書き戻し。

H28秋 問18：LRU・枠3。ページフォールトは何回？

参照 0, 1, 2, 3, 4, 0, 2, 4, 3, 1, 4, 5

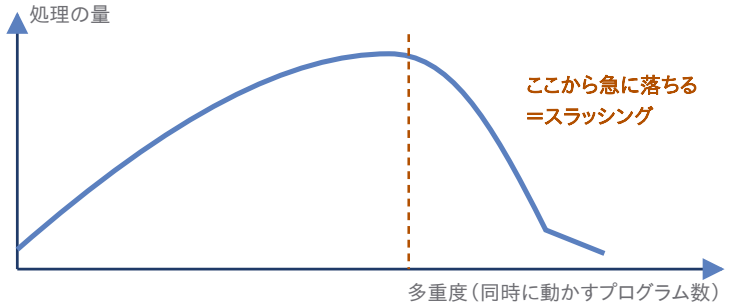
参照	結果	参照	結果
0・1・2	フォールト3回	4	あり
3	0を出す	3	0を出す
4	1を出す	1	2を出す
0	2を出す	4	あり
2	3を出す	5	3を出す

当たり（あり）は2回だけ。12回の参照のうち**10回**がフォールト（正解 イ）。最初の3回も**フォールトには数える**。

出典：IPA 平成24年度秋期 午前 問19・平成28年度秋期 午前 問18（参照列はClaudeがプログラムでも確かめた）

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

♻ ページの出し入ればかりで急に遅くなるのがスラッシング。多重度を下げて防ぐ



R3秋 問16・R6秋 問17（同じ問題）：「ページの置換えが頻繁に起きて、処理能力が急に落ちる」のは**スラッシング**。スワップアウト・フラグメンテーション・ページフォールトは、それぞれ1つの出来事や状態の名前。

R7秋 問19：キャッシュの表（読み込んだ時刻・最後に参照した時刻・参照回数）

選択肢	表のどの列で選ぶか
FIFO	読み込んだ時刻が**最も古い**もの
LFU	参照回数が**最も少ない**もの
LIFO	読み込んだ時刻が**最も新しい**もの
LRU	最後に参照した時刻が**最も古い**もの（この問題の答え）

キャッシュメモリの置換え（#3の範囲）も、ページ置換えと同じ名前と考え方。**列を1つずつ当てはめて**、C2が選ばれる方式を探す。

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

参照は近くに集まる（局所性）。だから必要な分だけ読む方式が効く

読み込み方	いつ読むか	よい所・弱い所
デマンドページング	フォールトが起きた**ときに**、そのページだけ	無駄なページを読まない。最初はフォールトが多い
プリページング	使いそうなページを**前もって**	当たれば補助記憶を待つ遅れが減る。外れると無駄

過去問 H23秋 問21：デマンドページングは「ページフォールトが発生したときに、当該ページを主記憶に読み込む」。H27秋 問17：利点は「アクセスされたときだけロードするので、**無駄なページをロードしなくて済む**」。R2秋 問18：プリページングの特徴は「前もってロードするので、**補助記憶へのアクセスによる遅れを減少**できる」。

言葉	意味
局所参照性	短い時間に、近い番地を繰り返し参照する性質。ループが典型
ワーキングセット	ある時間内に実際に参照したページの集まり。これが主記憶に収まっていればフォールトは少ない
スラッシング	ワーキングセットが入りきらず、出し入ればかりになる状態

H22春 問19：局所参照性が高いのは「ループによる反復のように、短い時間にメモリの**近接した場所**を参照する」プログラム。広い範囲を飛び回る分岐が多いほど低い。

出典：IPA 平成23年度秋期 午前 問21・平成27年度秋期 午前 問17・令和2年度10月 午前 問18・平成22年度春期 午前 問19／シラバス Ver.7.2 中分類5 1.(7)②の用語例（デマンドページング・ワーキングセット）

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

第4章

ミドルウェアとファイルシステム

ミドルウェア ／ API と ライブラリ ／ シェル ／ ディレクトリとパス ／ リンク ／ バックアップ ／ ファイル編成



詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

🤝 ミドルウェアは多くのアプリに共通の機能。OSとアプリの間に入る

ミドルウェア	受け持つこと
DBMS	データベースの定義・操作・同時実行の制御（#9）
TPモニター	大量のトランザクションを受け付け、処理を振り分けて完了まで管理する
アプリケーションサーバ	Webアプリの業務処理を動かす土台
メッセージキュー（MOM）	アプリ間でメッセージを**ためて非同期に**受け渡す
運用管理ツール	ジョブの自動実行・稼働の監視・ログの収集
ORB（CORBA）	ネットワーク上の別のオブジェクトを呼び出す



ミドルウェアを使うと、アプリは**OSの違いを気にせず**作れて、開発の手間も減る。Webサーバ・DBMS・アプリケーションサーバの3層は、#4の3層クライアントサーバと対応する。

詳しく：応用情報.com 用語集 5-2 ミドルウェア ／ 応用情報.com 用語集 5-1 オペレーティングシステム

🧩 部品を組み立てるのがコンポーネントウェア、土台ごと借りるのがフレームワーク

コンポーネント技術	中身
JavaBeans	Java の再利用できる部品の仕様
EJB	Enterprise JavaBeans。**サーバ側**の業務部品の仕様。トランザクションなどはサーバが面倒を見る
CORBA	言語や機械が違うオブジェクトを、ネットワーク越しに呼び出す標準
OSGi	Java の部品（バンドル）を、動かしたまま追加・入替えできる仕組み。Eclipse の土台

過去問 H29春 問17：サーバアプリ開発のための、オブジェクト指向に基づく**コンポーネントの仕様**は EJB。誤りの EAI は**システム間の連携**、ERP は**統合業務パッケージ**、UML は**モデルの表記法**。

開発フレームワーク	言語	特徴
Apache Struts	Java	MVC の考え方で作る Java の Web フレームワーク
Spring Framework	Java	DI（依存性の注入）で部品をつなぐ
Ruby on Rails	Ruby	「設定より規約」。決まりに従えば設定が少ない
CakePHP	PHP	Rails の考え方を PHP で

ライブラリは**自分のコードが呼ぶ**部品。フレームワークは**フレームワークが自分のコードを呼ぶ**（制御の反転）。全体の流れを決めるのがどちらか、で見分ける。

Hadoop は大量データを多数のサーバで**分散処理**するミドルウェア（H29秋 問19・R3春 問20）。

出典：IPA 平成29年度春期 午前 問17／IPA シラバス Ver.7.2 中分類5 2.（5）（6）の用語例

詳しく：応用情報.com 用語集 2-4 プログラム言語

🔗 APIは呼び出し方の約束。ライブラリは「いつつなぐか」で静的と動的

言葉	意味
API	ソフトウェアの機能を呼び出すための**決まりごと**（関数名・引数・戻り値）。ソースコードの段階の約束
ABI	機械語の段階の約束（呼出し規約・データの並べ方）。同じなら作り直さずに動く
Web API	HTTPで呼び出すAPI。REST方式が多い
OpenAPI	Web APIの仕様を書く形式。R7春 問17：それに沿って定義・開発を支えるOSSは**Swagger**

> シェル（bashなど）は、利用者が打ったコマンドを解釈してOSに伝える**コマンドインタプリタ**。複数のコマンドを並べたファイルが**シェルスクリプト**。

	静的リンク	動的リンク
つなぐ時	実行ファイルを作る時（リンカ）	**実行する時**（DLL・共有ライブラリ）
実行ファイル	ライブラリ入りで大きい	小さい。複数のプログラムで共用
ライブラリの更新	作り直しが必要	差し替えるだけで反映

> ライブラリは**自分のコードから呼ぶ**部品。フレームワークは土台の側が**自分のコードを呼ぶ**（制御の反転）。

詳しく：応用情報.com 用語集 5-2 ミドルウェア ／ 応用情報.com 用語集 5-1 オペレーティングシステム

ライブラリを「作るとき」につなぐか「動かすとき」につなぐか

静的リンク：リンカが実行ファイルに写し込む



動的リンク：起動時（または実行中）につなぐ



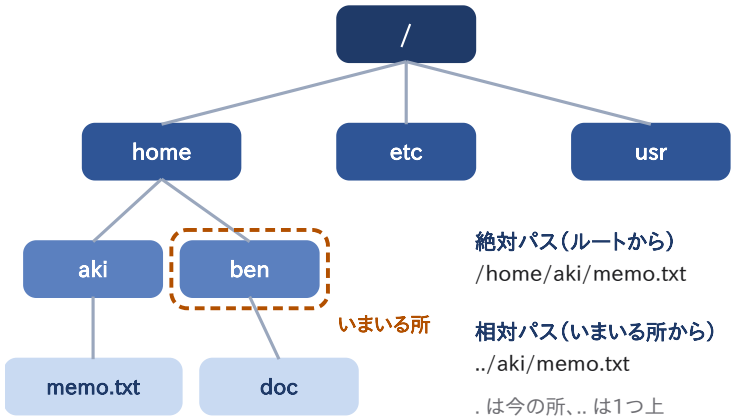
	静的リンク	動的リンク
つなぐ時	作るとき（リンク時）	動かすとき（ロード時・実行時）
大きさ	大きい	小さい。主記憶も**共有**できる
ライブラリの修正	作り直しが要る	ライブラリを差し替えるだけ
注意	環境が違っても動きやすい	版の食違いで動かない（DLL 地獄）

ライブラリの種類	入っているもの
ソースライブラリ	原始プログラム・マクロ
オブジェクトライブラリ	翻訳済みの目的プログラム
ロードライブラリ	すぐ実行できるロードモジュール
クラスライブラリ	オブジェクト指向の部品（クラス）

出典：Linux man-pages「ld.so(8)」（動的リンカが共有ライブラリを探して載せる）／IPA シラバス Ver.7.2 中分類5 2.（4）の用語例

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

ファイルは木の形で管理する。パスはルートからか、今いる所から書く



絶対パス（ルートから）
/home/aki/memo.txt

相対パス（いまいる所から）
../aki/memo.txt

. は今の所、.. は1つ上

リンク	指しているもの	元を消すと
ハードリンク	同じ**ファイルの実体**（iノード）にもう1つ名前を付ける	残りの名前から**まだ読める**
シンボリックリンク	元のファイルの**パス名**を中身に持つ別ファイル（ショートカット）	指す先がなくなり**読めない**

> シンボリックリンクはディレクトリや別のファイルシステムも指せる。ハードリンクは同じファイルシステムの中だけ。

> アクセス権（UNIX）：所有者・グループ・その他 ごとに**読み r・書き w・実行 x**。`rwxr-x---` は所有者が全部、グループが読みと実行、その他は何もできない。

詳しく：応用情報.com 用語集 5-3 ファイルシステム

UNIXのディレクトリは「名前→iノード番号」の表。中身の情報はiノードにある



2つの名前が同じ501を指す
＝ハードリンク

過去問 H21秋 問20：UNIX のファイルは通常・ディレクトリ・特殊の3種類。ディレクトリファイルは「**ファイル名とファイルの実体を対応付ける**ためのファイル」。

誤りの選択肢	本当は
入出力装置にアクセスするためのファイル	**特殊ファイル**（/dev の下。キャラクター型・ブロック型）
テキスト・オブジェクトコード・画像を入れる	**通常ファイル**
複数のパスから1つのファイルを参照する	**リンク**の説明

rm で名前を消すと、iノードのリンク数が1減る。**0になったとき**に初めて中身が消える。ハードリンクの片方を消しても読めるのはこのため。

出典：IPA 平成21年度秋期 午前 問20／Linux man-pages「inode(7)」（iノードの中身・リンク数）

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

🔑 アクセス権は「所有者・グループ・その他」に rwx。8進数1けたで3ビット

r=4　w=2　x=1　を足す

数字	表示	意味（ファイルの場合）
755	`rwxr-xr-x`	自分は全部、他人は読む・実行だけ（プログラム）
644	`rw-r--r--`	自分は読み書き、他人は読むだけ（ふつうの文書）
600	`rw-------`	自分だけ読み書き（秘密鍵など）
754	`rwxr-xr--`	7=4+2+1、5=4+1、4=4

例題 グループに書込みだけ足したい：`chmod g+w memo.txt`。644 なら **664**（rw-rw-r--）になる。

権限	ファイルでは	ディレクトリでは
r	中身を読む	中の**名前の一覧**を見る
w	中身を書き換える	中にファイルを**作る・消す・名前を変える**
x	プログラムとして実行	そのディレクトリに**入る**（中を通る）

ファイルを消せるかは、ファイル自身の w ではなく、**入っているディレクトリの w**で決まる。ディレクトリが名前の表だから。

Windows（NTFS）は利用者ごとに細かく許可を並べる **ACL**（アクセス制御リスト）で管理する。

出典：Linux man-pages「chmod(1)」／IPA シラバス Ver.7.2 中分類5 3.（1）の用語例（保護情報・アクセス権）

詳しく：応用情報.com 用語集 5-3 ファイルシステム

ファイルシステムはOSと用途で選ぶ。USBメモリのFAT32は1ファイル4GB未満

名前	主なOS・場所	特徴
FAT32	USBメモリ・SDカード	構造が簡単でどの機器でも読める。1ファイルは最大 **4GB−1バイト**。アクセス権がない
exFAT	大容量のSDカード	FATの後継。4GBを超えるファイルも置ける
NTFS	Windows	ACLによるアクセス権・ジャーナリング（更新を記録し、障害後に整合をとる）・暗号化
ReFS	Windows Server	データの**整合性**を重視。チェックサム（整合性ストリーム）で破損を見つけ、スキャンで直す
ext4・XFS	Linux	ジャーナリング。iノードで管理
NFS	ネットワーク	別のコンピュータのディスクを、**自分のディレクトリのように**使う（ファイルの共有）

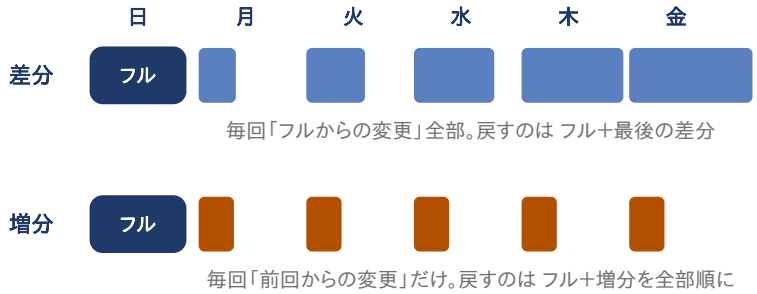
よくある困りごと 動画（5GB）を FAT32 の USBメモリへ写そうとすると、空きが十分でも「大きすぎる」と断られる。exFAT か NTFS でフォーマットし直せば置ける。

ボリューム＝1つのファイルシステムとして使う記憶領域のまとまり。1台のディスクを複数のボリュームに分けることも、複数台を1つにまとめることもある。

出典：Wikipedia「File Allocation Table」（最大ファイルサイズ 4,294,967,295バイト）／Microsoft Learn「ReFS の概要」／シラバス Ver.7.2 中分類5 3.（2）

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

増分は取るのが速く戻すのが遅い。差分はその中間



方式	取る時間・容量	戻す手間
フル	最大	1つ戻すだけ
差分	日がたつほど増える	フル＋最新の差分の**2つ**
増分	最小	フル＋増分**全部**（1つ欠けると戻せない）

ファイルの編成	しくみ
順編成	先頭から順に並べる。順に読むのは速い
直接編成	キーから**計算で**位置を決める（ハッシュ）。1件を探すのが速い
索引順編成	順に並べた本体＋キーの**索引**。順にも1件ずつにも読める
区分編成	複数の順編成のメンバと、その場所を示す登録簿。プログラムの保管

ファイルシステムの種類：WindowsのNTFS、Linuxのext4、USBメモリ向けのFAT32・exFAT。書込みの記録（ジャーナル）を先に残す**ジャーナリング**で、停電後も壊れにくくする。



詳しく：応用情報.com 用語集 5-3 ファイルシステム

「空きが最大の媒体を選ぶ」割当ては、使用量が最小の媒体を選ぶのと同じ

R1秋 問19：容量が同じ媒体A〜C。要求 90・30・40・40・70・30（Mバイト）。同じならA・B・Cの順

要求	A の使用量	B の使用量	C の使用量	選ぶ理由
90	**90**	0	0	全部空き→Aから
30	90	**30**	0	B・Cが同じ→B
40	90	30	**40**	Cが最小
40	90	**70**	40	Bが最小（30）
70	90	70	**110**	Cが最小（40）
30	90	**100**	110	Bが最小（70）

1. 容量が同じなので「空きが最大」＝「**使った量が最小**」。使用量の列だけ見ればよい
2. 最後の合計は A 90・B 100・C 110
3. 大きい順に **C, B, A**（正解 エ）

H22春 問20・H24秋 問20・H27秋 問18・R1秋 問19 の**4回**出た。数字を変えても、表の書き方は同じ。

同じ値のときの約束（A・B・Cの順）を読み落とすと、2番目の30をCに入れて結果が変わる。

出典：IPA 令和元年度秋期 午前 問19（平成22年度春期 問20・平成24年度秋期 問20・平成27年度秋期 問18も同じ問題）

詳しく：応用情報.com 用語集 5-3 ファイルシステム

レコードはまとめて1ブロックで読み書きする。何件入るかは割り算で決まる

物理レコード（ブロック）＝論理レコードをまとめたもの



ブロッキング＝まとめて1つにする／デブロッキング＝1件ずつに分ける

ブロック間の隙間（IBG）が減り、入出力の回数も減る

例題 1件200バイトのレコードを、1ブロック4,000バイトでまとめる。10,000件を入れるのに何ブロック要るか。

1. 1ブロックに 4,000÷200＝**20件**（ブロック化因数）
2. 10,000÷20＝**500ブロック**。読み込みも500回で済む（1件ずつなら10,000回）

ファイル編成	しくみ	アクセス
順編成	先頭から順に並べる	順次
直接編成	キーから**計算（ハッシュ）**で置き場所を決める。同じ場所になったら**あふれ域**へ	直接
索引順編成	順編成の本体＋**索引**。追加で入りきらない分はあふれ域へ	順次・直接の両方（動的アクセス）

割り切れないときは**切り捨て**（1件をブロックをまたいで置かない）。1件300バイトなら 4,000÷300＝13.3… → 1ブロック13件で、残りの100バイトは使わない。

出典：Claude が作った例題／IPA シラバス Ver.7.2 中分類5 3.（3）の用語例（論理レコード・物理レコード・ブロッキング・あふれ域）

詳しく：応用情報.com 用語集 9-2 データベース設計

差分と増分は「木曜の夜に壊れたら何本戻すか」で比べる

日曜にフル（100GB）。月〜土は毎日、別々のファイルが10GBずつ更新される

曜日	月	火	水	木	金	土	週の合計
差分（フルからの変更を全部）	10	20	30	40	50	60	**210GB**
増分（前回からの変更だけ）	10	10	10	10	10	10	**60GB**

木曜の夜（その日のバックアップ後）に壊れたときの戻し方	
差分	日曜のフル → **木曜の差分**（2本）
増分	日曜のフル → 月 → 火 → 水 → 木（**5本**を順番どおりに）

差分は日がたつほど取る量が増えるが、戻すのは2本で済む。
増分は毎日少ないが、**1本でも欠けると**その先が戻せない。

世代管理	考え方
多重バックアップ	複数世代を残す。最新が壊れていても1つ前から戻せる
祖父・父・子	日次（子）・週次（父）・月次（祖父）を決めた期間だけ残し、古いものから上書き

同じ場所にだけ置くと、火事や**ランサムウェア**でまとめて失う。別の場所・切り離した媒体にも残す。

出典：Claude が作った例題／IPA シラバス Ver.7.2 中分類5 3.（5）の用語例（多重バックアップ・フル・差分・増分バックアップ）

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

第5章

開発ツールとOSS

コンパイラの処理 ／ リンカとローダ ／ 言語処理の種類 ／ デバッグとテストの道具 ／ バージョン管理 ／ OSSのライセンス ／ 用途別のOSS



詳しく：応用情報.com 用語集 5-1 オペレーティングシステム

コンパイラは字句→構文→意味→最適化→コード生成。生成の直前が最適化

字句解析
文字の列を単語（トークン）に

▶

構文解析
文法どおりか・構文木

▶

意味解析
型や宣言の矛盾

▶

最適化
速く・小さく

▶

コード生成
目的プログラム

コンパイラ
原始→目的プログラム

▶

リンカ
→実行可能プログラム

▶

ローダ
主記憶に載せる

リンカ（連係編集）は目的プログラムとライブラリを**つなぐ**。ローダは実行可能プログラムを主記憶に**載せる**。載せる場所に合わせて番地を直すのが**再配置**。

R6秋 問20：コード生成の**直前**に行うのは**最適化**。字句解析は最初、構文解析と意味解析は途中。

最適化の例	中身
定数の畳込み	2*3 を先に 6 と計算しておく
ループ不変式の移動	ループの中で変わらない計算を外に出す
インライン展開	小さな関数の呼出しを中身で置き換える

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム ／ 応用情報.com 用語集 2-2 アルゴリズム

📜 どの段階で見つかる誤りかを、1行のプログラムで見分ける

プログラム（C言語風）	見つける段階	理由
`int 9x = 1;`	**字句解析**	数字で始まる名前は、単語（トークン）として作れない
`x = 1 + ;`	**構文解析**	単語は正しいが、文法（＋の後に式が要る）に合わない
`int x = "abc";`	**意味解析**	文法は正しいが、**型が合わない**
`y = z + 1;`（z を宣言していない）	**意味解析**	宣言と使用が**対応付かない**
`a[10]`（大きさ5の配列）	多くは**実行時**	添字が変数なら、翻訳時には値が分からない

過去問 H25秋 問20：意味解析で行うのは「**変数の宣言と使用とを対応付けたり、演算におけるデータ型の整合性をチェック**する」。

誤りの選択肢	どの段階か
文法に基づき解析し、文法誤りがないかチェック	構文解析
文字の列を、意味のある最小の構成要素の列に変換	字句解析
レジスタ割付けや、不要な演算を省くプログラム変換	最適化

出典：IPA 平成25年度秋期 午前 問20／Claude が作ったプログラム例

詳しく：応用情報.com 用語集 2-2 アルゴリズム

🛠️ 最適化は「結果を変えずに速くする」書換え。代表的な6つ

名前	書換え前	書換え後
定数の畳込み	`x = 60 * 60 * 24;`	`x = 86400;`
定数伝播	`n = 10; y = n * 2;`	`y = 20;`（n は変わらない）
共通部分式の削除	`p = a*b + c; q = a*b - d;`	`t = a*b; p = t + c; q = t - d;`
ループ不変式の移動	`for(…){ s += x * k / 2; }`	`h = k / 2; for(…){ s += x * h; }`
ループ展開	`for(i=0;i<4;i++) a[i]=0;`	`a[0]=0; a[1]=0; a[2]=0; a[3]=0;`
強さの軽減	`y = x * 2;`	`y = x + x;`（または左シフト）

ループ不変式の例は、x * h と書き換えても同じ結果になる整数の場合

過去問 H27秋 問19：実行時間を短くする最適化は「値が変わらない変数を**定数で置き換える**」（定数伝播）。

誤りの選択肢	なぜ違うか
10,000回のループを100回×100回にする	ループの管理が増え、遅くなる
X+X を 2*X に置き換える	**逆**。乗算の方が重いので、乗算を加算にする
同じ処理をサブルーチンにまとめる	小さくはなるが、呼出しの分だけ**遅くなる**（速くするのはインライン展開）

出典：IPA 平成27年度秋期 午前 問19／Claude が作ったプログラム例

詳しく：応用情報.com 用語集 5-4 開発ツール

別のCPU向けの機械語を作るのがクロスコンパイラ。組込み開発の基本

言語プロセッサ	何をするか
コンパイラ	原始プログラムを**まとめて**目的プログラムに翻訳
インタプリタ	1文ずつ解釈しながら**すぐ実行**。目的プログラムを作らない
アセンブラ	アセンブリ言語（機械語と1対1）を機械語に
クロスコンパイラ	動かすのと**異なる命令形式**のコンピュータ用の目的プログラムを作る
JITコンパイラ	実行中に、よく使う中間コードを機械語へ直す（Java・JavaScript）
プリプロセッサ	コンパイルの前に、マクロの展開やファイルの取込みを行う



R5秋 問18：「異なる命令形式のコンピュータで実行できる目的プログラムを生成する」のは**クロスコンパイラ**。エミュレーター（他の機械の**まね**をして動かす）、最適化コンパイラ、プログラムジェネレーター（仕様から**プログラムを自動生成**）と入れ替えた選択肢に注意。

開発するコンピュータを**ホスト**、動かす機器を**ターゲット**と呼ぶ。

詳しく：応用情報.com 用語集 5-4 開発ツール ／ 応用情報.com 用語集 5-1 オペレーティングシステム

回数と時間を測るのがプロファイラ。全履歴を手元に持つのがGit

道具	何をするか
プロファイラ	関数などの**実行回数・実行時間**を集め、遅い所を見つける
デバッガ	止めながら実行し、変数の中身を見て誤りを探す（ブレークポイント・トレース）
エミュレーター	別のハードウェアの動きを**まねて**、そのプログラムを動かす
シミュレーター	実物の代わりに、ソフトウェアで**動きを模擬**して確かめる
テストドライバ・スタブ	下位を試す時の**呼ぶ側の代わり**・上位を試す時の**呼ばれる側の代わり**（#12）
静的解析ツール	動かさずにソースコードを調べて誤りや規約違反を探す

R6春 問19：実行回数や実行時間を集めるのは**プロファイラ**。

	集中型	分散型
例	Subversion・CVS・RCS	**Git**・Mercurial
履歴の置き場所	サーバに1つ	各自の手元に**全履歴の完全な複製**
つながらない時	記録（コミット）できない	手元でコミットでき、あとで送る（push）

R3秋 問18・R6秋 問19（同じ問題）：「ローカルに全履歴の完全な複製を持つ」のは**Git**。ほか3つは集中型（RCSは1ファイル単位の古い道具）。

詳しく：応用情報.com 用語集 5-4 開発ツール ／ 応用情報.com 用語集 12-3 実装・構築

デバッグの道具は「動かさずに見る」か「動かして見る」かで分ける

動かさずに見る（静的）	
プリティプリンター	字下げをそろえて**読みやすく整形**する
クロスリファレンス	変数や手続きが**どこで定義・参照**されているかの一覧
静的解析ツール	書き方の誤り・規約違反・脆弱性を、実行せずに探す
動かして見る（動的）	
トレーサー	実行した**経路**（命令・手続きの順）を記録・表示
インスペクター	実行時の**データの中身**を表示
スナップショット	指定した時点の主記憶やレジスタの内容を出力
アサーションチェッカ	「ここでは必ず x＞0」などの条件を実行中に確かめる
テストカバレージ	テストで**実行された部分の割合**を測る
ICE	組込み機器のCPUの代わりに差し込み、実機の上で動きを観察する

過去問 H22春 問21：正しいのは「インスペクターは、プログラム実行時にデータ内容を表示する」。

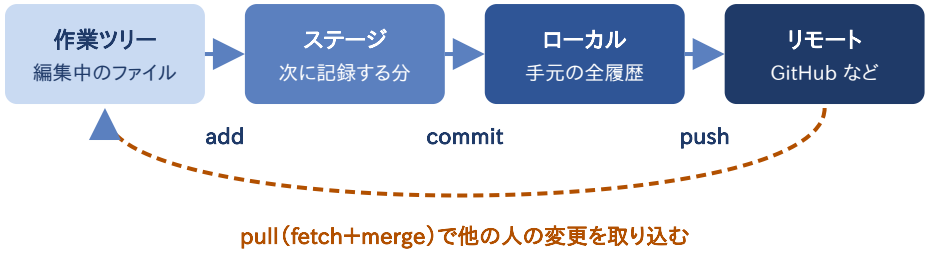
誤りの選択肢の説明	本当は何の道具か
実行経路を表示する（とされたシミュレーター）	**トレーサー**
機能説明やデータ定義の探索を容易にする（とされたトレーサー）	**ブラウザ**（ソースを見て回る道具）
文字の挿入・削除・置換で編集する（とされたブラウザ）	**エディタ**

過去問 H29春 問18：ホワイトボックステストで実行された部分の割合を測るのは**テストカバレージ分析ツール**。H30秋 問19：関数の実行回数や実行時間の統計を取るのは**プロファイラ**。

出典：IPA 平成22年度春期 午前 問21・平成29年度春期 午前 問18・平成30年度秋期 午前 問19／シラバス Ver.7.2 中分類5 4.（1）の用語例

詳しく：応用情報.com 用語集 5-4 開発ツール

📜 Gitは手元に全履歴を持つ分散型。add→commit→push の3段で送る



ブランチ：本流から枝分かれして作業し、終わったら merge で戻す
同じ行を2人が変えると衝突（コンフリクト）。人が選んで直す

コマンド	すること
`git clone URL`	リモートの履歴を**丸ごと**手元に複製する
`git add memo.txt`	次のコミットに入れる変更を選ぶ
`git commit -m "…"`	手元の履歴に1つ記録する（サーバ不要）
`git push`	手元の記録をリモートへ送る
`git pull`	リモートの変更を取り込む
`git branch／merge`	枝を作る／枝の変更を合流させる

> R3秋 問18・R6秋 問19（同じ問題）では「ローカルに**全履歴の完全な複製**を持つ」のが Git。集中型の Subversion・CVS と入れ替えて出る。

出典：Pro Git（日本語版）／IPA シラバス Ver.7.2 中分類5 4.（1）・5.（3）③の用語例（バージョン管理ツール・Subversion・Git）

詳しく：応用情報.com 用語集 5-5 オープンソースソフトウェア

📜 ソース公開の義務は GPL→LGPL→MPL→BSD の順に軽くなる

R7春 問16：改変して配るとき、ソースの公開が必要か

ライセンス	改変した部分	静的リンクした自分のコード	動的リンクした自分のコード
GPL	必要	必要	必要
LGPL	必要	必要（結合部分の再リンクを許す）	不要
MPL	必要（ファイル単位）	不要	不要
BSD・MIT・Apache	不要	不要	不要

表の〇の数が多い順に **GPL, LGPL, MPL, BSD**（ア）。GPLのように「派生物も同じ条件で配る」考え方を**コピーレフト**と呼ぶ。

OSI の定義	中身
自由な再配布	売ってもよい。ライセンス料を取らない
ソースコード	入手できるようにする
差別の禁止	人・団体・**利用分野**で制限しない（「商用は禁止」は不可）

OSSでも**著作権は放棄されていない**。無保証が原則で、ライセンスの条件を守る義務がある。

詳しく：応用情報.com 用語集 5-5 オープンソースソフトウェア ／ 応用情報.com 用語集 23-3 労働関連・取引関連法規

🤝 OSSの条件はOSIの定義10か条。「無料」も「著作権放棄」も条件ではない

#	The Open Source Definition	要点
1	再頒布の自由	売っても配ってもよい。使用料を求めてはならない
2	ソースコード	ソースを含めるか、入手方法を示す
3	派生物	改変と、同じ条件での再頒布を認める
4	作者のソースの完全性	改変はパッチで、という制限は許される
5	個人・集団の差別禁止	特定の人を排除しない
6	利用分野の差別禁止	**商用**や遺伝子研究などの分野で禁止しない
7	ライセンスの分配	受け取った人に、追加の契約なしで権利が及ぶ
8〜10	製品に特定しない・他のソフトを制限しない・技術中立	一緒に配る他のソフトにまで条件を押し付けない

過去問	正解の要点
H21秋 問21	**営利企業での使用や研究分野での使用も許可**される（第6条）
H22秋 問21	**一定の条件の下で**、ソースの変更を許可している
H23特別 問23	パッケージ化したり自社ソフトと組み合わせたりして**有償で販売**できる

> 誤りの選択肢の定番：「無償で配布しなければならない」「著作権が放棄されている」「利用者や分野を制限できる」「改良版は再頒布できない」「一緒に配る他のソフトもソース公開が必要」。どれも定義に反する。

出典：Open Source Initiative「The Open Source Definition」（Version 1.9）／IPA 平成21年度秋期 問21・平成22年度秋期 問21・平成23年度特別 問23

詳しく：応用情報.com 用語集 5-5 オープンソースソフトウェア

📜 ライセンスごとに「改変して配る人の義務」を、条文の言葉で押さえる

ライセンス	改変して配るときの義務（要点）	性格
GPL	作品**全体**をGPLで提供する（GPLv3 第5条 c）。ソースも出す	強いコピーレフト
LGPL	ライブラリ自体の改変はLGPLで公開。**リンクして使うだけ**のアプリは別のライセンスでよい	弱いコピーレフト
MPL 2.0	MPLのコードを含む**ファイル単位**で公開。他のファイルは自由	ファイル単位のコピーレフト
Apache 2.0	ライセンス文を添付し、**変更したファイルに変更した旨を目立つように示す**（第4条 b）	非コピーレフト
BSD・MIT	著作権表示とライセンス文を残す	非コピーレフト

> ソースを独占的に扱いたい企業向けに、同じソフトを**GPLと商用**の2つで出すのが**デュアルライセンス**。

過去問	正解
H22春 問22（Apacheを変更して配る）	変更したファイルに**変更した旨の告知**を入れる
H25春 問21（GPLを改変・頒布）	**同一ライセンス**での頒布
H29春 問19（GPLの説明）	GPLのソースの派生物は**無条件にGPL**になる

> 誤りの定番：Apache の名前を製品名に使う（商標は許されない）／GPLは無償で頒布（**有償でもよい**）／GPLの機能を呼ぶ別プログラムも全部GPL（独立したプログラムは対象外）。

出典：GPL-3.0 本文（OSI 掲載）／Apache License 2.0／MPL 2.0 FAQ／IPA 平成22年度春期 問22・平成25年度春期 問21・平成29年度春期 問19

詳しく：応用情報.com 用語集 5-5 オープンソースソフトウェア ／ 応用情報.com 用語集 23-3 労働関連・取引関連法規

Linuxはカーネルだけ。動く形に組んで配るのがディストリビュータ

アプリ（Web・メール・DNS）
Apache・Postfix・BIND

シェル・コマンド・ライブラリ
bash・GNU のツール・C ライブラリ

Linux カーネル
プロセス管理・記憶管理・ドライバ

これらをまとめ、導入プログラムや更新の仕組みを付けて配るのが**ディストリビューション**（Red Hat Enterprise Linux・Ubuntu・Debian など）。R2秋問19：ディストリビュータの役割は「OSSやアプリを組み合わせて、**パッケージにして提供**する」。

H21春 問21：サーバの種類とOSSの組合せ

サーバ	代表的なOSS	間違えやすい相手
DNS	**BIND**	Apache（Web）と入れ替えて出る
Web	**Apache**（HTTP Server）・nginx	Postfix（メール）
メール（SMTP）	**Postfix**・sendmail	BIND（DNS）

組合せ	中身
LAMP	Linux＋Apache＋MySQL＋PHP（Perl・Python）
LAPP	Linux＋Apache＋PostgreSQL＋PHP

言語ごとの部品置場：Perl は **CPAN**、PHP は **PEAR**、Python は PyPI。

出典：IPA 令和2年度10月 午前 問19・平成21年度春期 午前 問21（組合せの表を文字で書き直した）／シラバス Ver.7.2 中分類5 5.（1）（2）の用語例

詳しく：応用情報.com 用語集 5-5 オープンソースソフトウェア ／ 応用情報.com 用語集 5-1 オペレーティングシステム

OSSは用途とセットで覚える。似た分野の名前と入れ替えて出る

用途	代表的なOSS	過去問で問われ方
OS	Linux・FreeBSD・Android	R5秋19：Linuxカーネルの役目
仮想化・コンテナ	**Docker**（コンテナ）・Kubernetes（コンテナの管理）・KVM・Xen・QEMU（仮想マシン）	R5春20：コンテナ型仮想化は**Docker**
大規模データ	**Hadoop**（分散処理）・Spark	R3春20：大量データを分散処理するライブラリ
Web・API	Apache HTTP Server・nginx・Tomcat（アプリサーバ）・**Swagger**（API定義）・curl	R7春17：OpenAPIに従った定義はSwagger
DBMS	MySQL・PostgreSQL・SQLite・Redis・MongoDB	—
運用・構成管理	Ansible・Chef（構成の自動化）・Serverspec（構成のテスト）・Zabbix（監視）・Fluentd（ログ収集）	Serverspecは選択肢に登場
認証	OpenAM・Keycloak（シングルサインオン）	OpenAMは選択肢に登場
開発	Git・Jenkins（CI）・Eclipse・GCC（コンパイラ）	R3秋18：Git



詳しく：応用情報.com 用語集 5-5 オープンソースソフトウェア ／ 応用情報.com 用語集 9-1 データベース方式

OSSを支える団体と、使う側が負う責任

団体	役目
OSI（Open Source Initiative）	オープンソースの定義（OSD）を管理し、ライセンスを認定する
FSF（Free Software Foundation）	「自由なソフトウェア」を広める。GNU と GPL を作った
Linux Foundation	Linux カーネルや Kubernetes（CNCF）などの開発を支える
Apache Software Foundation	Apache HTTP Server・Hadoop など多数のプロジェクトを運営
OpenSSF	OSSのセキュリティ（供給網の安全）を高める活動。Linux Foundation の下

考慮点	中身
サポート	メーカーの保証がない。有償サポート契約や社内の技術者で補う
脆弱性への対応	使っているOSSと版を一覧にして（SBOM）、更新の情報を追う
ライセンスの順守	製品に組み込んで配る前に、義務（ソース公開・表示）を確かめる
継続性	開発が止まることがある（例：Chainer は2019年に開発の中心を PyTorch へ移した）

> シラバスの用語例は「安全性・信頼性」。**無償＝費用ゼロではない**。保守・調査の手間を含めた総費用で判断する。

出典：IPA シラバス Ver.7.2 中分類5 5.（4）（5）の用語例／OSI「About」／FSF「About」／Preferred Networks「深層学習の研究開発基盤をPyTorchに移行」（2019年12月5日）

詳しく：応用情報.com 用語集 5-5 オープンソースソフトウェア

📋 まとめ：表と線を自分で書けるようにすれば、128問の論点はほぼ取れる

論点	覚えること	過去問
状態遷移	横取り→実行可能。入出力の開始→待ち。待ち→実行は直接移らない	R3春17・R7春14・R6春16
割込みとRTOS	割込み処理は最高優先度のタスクより先。周期タスクは高い方の使用時間を引く	R4秋18・R5秋17
時間の計算	線を引く。ターンアラウンド＝終了－到着。多重度は到着のずれに注意	R3春16・R7秋15・R4春19・R6春18
スケジューリング	SJFはスタベーション、エージングで防ぐ	R6秋16
排他制御	共有資源はセマフォ。デッドロックは取る順番をそろえる。スレッドセーフ	R4春17・R7秋16・R4秋16・R6春17・R7秋17
記憶管理	断片化＝連続した空きがない。ページングは外部断片化なし。GC	R3春18・R4春18・R6秋18・R4秋17
仮想記憶	PF＝PI≧PO。FIFO＝読込み時刻、LRU＝最終使用時刻。スラッシング	R3春19・R5春17・R7春15・R3秋16 ほか
開発ツール・OSS	最適化はコード生成の直前。クロスコンパイラ・プロファイラ・Git。GPL→BSD	R6秋20・R5秋18・R6春19・R7春16 ほか

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム ／ 応用情報.com 用語集 5-5 オープンソースソフトウェア

⚖️ 試験で狙われやすい比較10選

#	比べるもの	見分けるポイント
1	プリエンプティブ／ノンプリエンプティブ	OSが横取りできるか。到着順（FCFS）は横取りしない
2	実行可能状態／待ち状態	CPUさえあれば動ける／入出力などを待っていて動けない
3	ディスパッチ／スケジューリング	CPUを渡すこと／順番を決めること
4	プロセス／スレッド	メモリ別々／プロセスの中でメモリを共有（スタックは別）
5	内部断片化／外部断片化	区画の中の余り（固定区画）／区画の間の細かい空き（可変区画）
6	ページング／セグメンテーション	固定長（外部断片化なし）／論理的なまとまりで可変長
7	FIFO／LRU／LFU	読み込んだ時刻／最後に使った時刻／使った回数
8	デマンドページング／プリページング	フォールトの時に読む／前もって読む
9	ハードリンク／シンボリックリンク	同じiノードに別の名前（元を消しても読める）／パス名を持つ別ファイル（読めなくなる）
10	GPL／LGPL／MPL／Apache・BSD	全体をGPL／リンクだけなら自由／ファイル単位／表示を残せば自由（Apacheは変更の告知も）

出典：この資料の各ページ（IPA 過去問題 平成21〜令和7年度・シラバス Ver.7.2）

詳しく：応用情報.com 用語集 5-1 オペレーティングシステム ／ 応用情報.com 用語集 5-3 ファイルシステム

出典（1）IPAの資料と過去問

資料	使ったところ
IPA「応用情報技術者試験（レベル3）シラバス Ver.7.2」	中分類5 ソフトウェアの目標と用語例（各章の用語の範囲）
IPA 応用情報技術者試験 午前 過去問題（平成21年度春期～令和2年度10月）	例題・過去問の箱。この分野の86問のうち、令和3年度以降と重ならない問題を問題演習に追加
IPA 応用情報技術者試験 午前 過去問題（令和3年度春期～令和7年度秋期）	元からあるページの過去問（42問）

過去問の問題文はIPAの公開資料（許諾・使用料不要）を使った。図や表の問題は、画像を見て文字と図に書き直した。解説はClaudeが自分の言葉で書いた。

「Claude が作った例題」は、過去問と同じ型の計算を、数字を変えて作ったもの。答えは手計算とプログラムの両方で確かめた。

IPA 過去問題：https://www.ipa.go.jp/shiken/mondai-kaiotu/　／　シラバス：https://www.ipa.go.jp/shiken/syllabus/

出典（2）技術資料・実例（2026年11月6日に確認）

資料	使ったところ
Linux man-pages：syscalls(2)・pthreads(7)・inode(7)・chmod(1)・crontab(5)・ld.so(8)	システムコール、スレッドが共有するもの、iノード、アクセス権、cron、動的リンク
Linux Kernel Documentation「Page Tables」	4KBのページ、最初の Linux の表（1,024項目・4MB）、5段の表、ヒュージページ
The Open Group「POSIX.1-2024」	POSIX の位置付け
Open Source Initiative「The Open Source Definition」・GPL-3.0 本文、Apache License 2.0、MPL 2.0 FAQ	OSDの10か条、ライセンスの義務
Wikipedia：Mars Pathfinder／Priority inversion／Bélády's anomaly／Deadlock／Multilevel feedback queue／Semaphore／Bell–LaPadula model／File Allocation Table／TLB／Reentrancy／Memory-mapped I/O	実例と定義の確認
Microsoft Learn「ReFS の概要」「ユーザー アカウント制御」	ReFS の整合性ストリーム、UAC
Python ドキュメント「gc」、Pro Git、Preferred Networks（2019年12月5日）、TOPPERS、OSI・FSF の団体紹介	GC、Git、Chainer の移行、μITRON、OSS団体

読めなかった資料：GNU の GPL 本文と FAQ（gnu.org に接続できず、OSI 掲載の GPL-3.0 本文で確認）、NASA/JPL のパスファインダーの報告（Wikipedia の記述と出典表示で確認）